

Concurrency

- 13.1** Introduction
- 13.2** Introduction to Subprogram-Level Concurrency
- 13.3** Semaphores
- 13.4** Monitors
- 13.5** Message Passing
- 13.6** Ada Support for Concurrency
- 13.7** Java Threads
- 13.8** C# Threads
- 13.9** Concurrency in Functional Languages
- 13.10** Statement-Level Concurrency

This chapter begins with introductions to the various kinds of concurrency at the subprogram, or unit level, and at the statement level. Included is a brief description of the most common kinds of multiprocessor computer architectures. Next, a lengthy discussion on unit-level concurrency is presented. This begins with a description of the fundamental concepts that must be understood before discussing the problems and challenges of language support for unit-level concurrency, specifically competition and cooperation synchronization. Next, the design issues for providing language support for concurrency are described. Following this is a detailed discussion of three major approaches to language support for concurrency: semaphores, monitors, and message passing. A pseudocode example program is used to demonstrate how semaphores can be used. Ada and Java are used to illustrate monitors; for message passing, Ada is used. The Ada features that support concurrency are described in some detail. Although tasks are the focus, protected objects (which are effectively monitors) are also discussed. Support for unit-level concurrency using threads in Java and C# is then discussed, including approaches to synchronization. This is followed by brief overviews of support for concurrency in several functional programming languages. The last section of the chapter is a brief discussion of statement-level concurrency, including an introduction to part of the language support provided for it in High-Performance Fortran.

13.1 Introduction

Concurrency in software execution can occur at four different levels: instruction level (executing two or more machine instructions simultaneously), statement level (executing two or more high-level language statements simultaneously), unit level (executing two or more subprogram units simultaneously), and program level (executing two or more programs simultaneously). Because no language design issues are involved with them, instruction-level and program-level concurrency are not discussed in this chapter. Concurrency at both the subprogram and the statement levels is discussed, with most of the focus on the subprogram level.

At first glance, concurrency may appear to be a simple concept, but it presents significant challenges to the programmer, the programming language designer, and the operating system designer (because much of the support for concurrency is provided by the operating system).

Concurrent control mechanisms increase programming flexibility. They were originally invented to be used for particular problems faced in operating systems, but they are required for a variety of other programming applications. One of the most commonly used programs is Web browsers, whose design is based heavily on concurrency. Browsers must perform many different functions at the same time, among them sending and receiving data from Web servers, rendering text and images on the screen, and reacting to user actions with the mouse and the keyboard. Most contemporary browsers use the extra core processors that are part of many contemporary personal computers to perform some of their processing, for example the interpretation of client-side scripting

code. Another example is the software systems that are designed to simulate actual physical systems that consist of multiple concurrent subsystems. For all of these kinds of applications, the programming language (or a library or at least the operating system) must support unit-level concurrency.

Statement-level concurrency is quite different from concurrency at the unit level. From a language designer's point of view, statement-level concurrency is largely a matter of specifying how data should be distributed over multiple memories and which statements can be executed concurrently.

The goal of developing concurrent software is to produce scalable and portable concurrent algorithms. A concurrent algorithm is **scalable** if the speed of its execution increases when more processors are available. This is important because the number of processors sometimes increases with the new generations of machines. The algorithms must be portable because the lifetime of hardware is relatively short. Therefore, software systems should not depend on a particular architecture—that is, they should run efficiently on machines with different architectures.

The intention of this chapter is to discuss the aspects of concurrency that are most relevant to language design issues, rather than to present a definitive study of all of the issues of concurrency, including the development of concurrent programs. That would clearly be inappropriate for a book on programming languages.

13.1.1 Multiprocessor Architectures

A large number of different computer architectures have more than one processor and can support some form of concurrent execution. Before beginning to discuss concurrent execution of programs and statements, we briefly describe some of these architectures.

The first computers that had multiple processors had one general-purpose processor and one or more other processors, often called peripheral processors, that were used only for input and output operations. This architecture allowed those computers, which appeared in the late 1950s, to execute one program while concurrently performing input or output for that program or other programs.

By the early 1960s, there were machines that had multiple complete processors. These processors were used by the job scheduler of the operating system, which distributed separate jobs from a batch-job queue to the separate processors. Systems with this structure supported program-level concurrency.

In the mid-1960s, some machines appeared that had several identical partial processors. These were fed instructions from a single instruction stream. For example, some machines had two or more floating-point multipliers, while others had two or more complete floating-point arithmetic units. The compilers for these machines were required to determine which instructions could be executed concurrently and to schedule these instructions accordingly. Systems with this structure supported instruction-level concurrency.

In 1966, Michael J. Flynn suggested a categorization of computer architectures defined by whether the instruction and data streams were single or multiple. The names of these were widely used from the 1970s to the early 2000s. The two categories that used multiple data streams are defined as follows: Computers that have multiple processors that execute the same instruction simultaneously, each on different data, are called Single Instruction, Multiple Data (SIMD) architecture computers. In an SIMD computer, each processor has its own local memory. One processor controls the operation of the other processors. Because all of the processors, except the controller, execute the same instruction at the same time, no synchronization is required in the software. Perhaps the most widely used SIMD machines are a category of machines called **vector processors**. They have groups of registers that store the operands of a vector operation in which the same instruction is executed on the whole group of operands simultaneously. Originally, the kinds of programs that could most benefit from this architecture were in scientific computation, an area of computing that is often the target of multiprocessor machines. However, SIMD processors are now used for a variety of application areas, among them graphics and video processing. Until recently, most supercomputers were vector processors.

Computers that have multiple processors that operate independently but whose operations can be synchronized are called Multiple Instruction, Multiple Data (MIMD) computers. Each processor in an MIMD computer executes its own instruction stream. MIMD computers can appear in two distinct configurations: distributed and shared memory systems. The distributed MIMD machines, in which each processor has its own memory, can be either built in a single chassis or distributed, perhaps over a large area. The shared-memory MIMD machines obviously must provide some means of synchronization to prevent memory access clashes. Even distributed MIMD machines require synchronization to operate together on single programs. MIMD computers, which are more general than SIMD computers, support unit-level concurrency. The primary focus of this chapter is on language design for shared memory MIMD computers, which are often called **multiprocessors**.

With the advent of powerful but low-cost single-chip computers, it became possible to have large numbers of these microprocessors connected into physically small networks within a single chassis. These kinds of computers, which often use off-the-shelf microprocessors, are available from a number of different manufacturers.

One important reason why software has not evolved faster to make use of concurrent machines is that the power of processors has continually increased. One of the strongest motivations to use concurrent machines is to increase the speed of computation. However, two hardware factors have combined to provide faster computation, without requiring any change in the architecture of software systems. First, processor clock rates have become faster with each new generation of processors (the generations have appeared roughly every 18 months). Second, several different kinds of concurrency have been built into the processor architectures. Among these are the pipelining of instructions and data from the memory to the processor (instructions are

fetches and decodes for future execution while the current instruction is being executed), the use of separate lines for instructions and data, prefetching of instructions and data, and parallelism in the execution of arithmetic operations. All of these are collectively called **hidden concurrency**. The result of the increases in execution speed is that there have been great productivity gains without requiring software developers to produce concurrent software systems.

However, the situation has changed. The end of the sequence of significant increases in the speed of individual processors is now near. Significant increases in computing power now result from increases in the number of processors, for example large server systems like those run by Google and Amazon and scientific research applications. Many other large computing tasks are now run on machines with large numbers of relatively small processors.

Another recent advance in computing hardware was the development of multiple processors on a single chip, such as with the Intel Core Duo and Core Quad chips, which is putting more pressure on software developers to make more use of the available multiple processor machines. If they do not, the concurrent hardware will be wasted and significant productivity gains will not be realized.

13.1.2 Categories of Concurrency

There are two distinct categories of concurrent unit control. The most natural category of concurrency is that in which, assuming that more than one processor is available, several program units from the same program literally execute simultaneously. This is **physical concurrency**. A slight relaxation of this concept of concurrency allows the programmer and the application software to assume that there are multiple processors providing actual concurrency, when in fact the actual execution of programs is taking place in interleaved fashion on a single processor. This is **logical concurrency**. From the programmer's and language designer's points of view, logical concurrency is the same as physical concurrency. It is the language implementor's task, using the capabilities of the underlying operating system, to map the logical concurrency to the host hardware. Both logical and physical concurrencies allow the concept of concurrency to be used as a program design methodology. For the remainder of this chapter, the discussion will apply to both physical and logical concurrencies.

One useful technique for visualizing the flow of execution through a program is to imagine a thread laid on the statements of the source text of the program. Every statement reached on a particular execution is covered by the thread representing that execution. Visually following the thread through the source program traces the execution flow through the executable version of the program. Of course, in all but the simplest of programs, the thread follows a highly complex path that would be impossible to follow visually. Formally, a **thread of control** in a program is the sequence of program points reached as control flows through the program.

Programs that have coroutines (see Chapter 9) but no concurrent subprograms, though they are sometimes called **quasi-concurrent**, have a single

thread of control. Programs executed with physical concurrency can have multiple threads of control. Each processor can execute one of the threads. Although logically concurrent program execution may actually have only a single thread of control, such programs can be designed and analyzed only by imagining them as having multiple threads of control. A program designed to have more than one thread of control is said to be **multithreaded**. When a multithreaded program executes on a single-processor machine, its threads are mapped onto a single thread. It becomes, in this scenario, a virtually multithreaded program.

Statement-level concurrency is a relatively simple concept. In a common use of statement-level concurrency, loops that include statements that operate on array elements are unwound so that the processing can be distributed over multiple processors. For example, a loop that executes 500 repetitions and includes a statement that operates on one of 500 array elements may be unwound so that each of 10 different processors can simultaneously process 50 of the array elements.

13.1.3 Motivations for the Use of Concurrency

There are at least four different reasons to design concurrent software systems. The first reason is the speed of execution of programs on machines with multiple processors. These machines provide an effective way of increasing the execution speed of programs, provided that the programs are designed to make use of the concurrent hardware. There are now a large number of installed multiple-processor computers, including many of the personal computers sold in the last few years. It is wasteful not to use this hardware capability.

The second reason is that even when a machine has just one processor, a program written to use concurrent execution can be faster than the same program written for sequential (nonconcurrent) execution. The requirement for this to happen is that the program is not compute bound (the sequential version does not fully utilize the processor).

The third reason is that concurrency provides a different method of conceptualizing program solutions to problems. Many problem domains lend themselves naturally to concurrency in much the same way that recursion is a natural way to design solutions to some problems. Also, many programs are written to simulate physical entities and activities. In many cases, the system being simulated includes more than one entity, and the entities do whatever they do simultaneously—for example, aircraft flying in a controlled airspace, relay stations in a communications network, and the various machines in a factory. Software that uses concurrency must be used to simulate such systems accurately.

The fourth reason for using concurrency is to program applications that are distributed over several machines, either locally or through the Internet. Many machines, for example cars, have more than one built-in computer, each of which is dedicated to some specific task. In many cases, these collections of computers must synchronize their program executions. Internet games are another example of software that is distributed over multiple processors.

Concurrency is now used in numerous everyday computing tasks. Web servers process document requests concurrently. Web browsers now use secondary core processors to run graphic processing and to interpret programming code embedded in documents. In every operating system there are many concurrent processes being executed at all times, managing resources, getting input from keyboards, displaying output from programs, and reading and writing external memory devices. In short, concurrency has become a ubiquitous part of computing.

13.2 Introduction to Subprogram-Level Concurrency

Before language support for concurrency can be considered, one must understand the underlying concepts of concurrency and the requirements for it to be useful. These topics are covered in this section.

13.2.1 Fundamental Concepts

A **task** is a unit of a program, similar to a subprogram, that can be in concurrent execution with other units of the same program. Each task in a program can support one thread of control. Tasks are sometimes called **processes**. In some languages, for example Java and C#, certain methods serve as tasks. Such methods are executed in objects called **threads**.

Three characteristics of tasks distinguish them from subprograms. First, a task may be implicitly started, whereas a subprogram must be explicitly called. Second, when a program unit invokes a task, in some cases it need not wait for the task to complete its execution before continuing its own. Third, when the execution of a task is completed, control may or may not return to the unit that started that execution.

Tasks fall into two general categories: heavyweight and lightweight. Simply stated, a **heavyweight task** executes in its own address space. **Lightweight tasks** all run in the same address space. It is easier to implement lightweight tasks than heavyweight tasks. Furthermore, lightweight tasks can be more efficient than heavyweight tasks, because less effort is required to manage their execution.

A task can communicate with other tasks through shared nonlocal variables, through message passing, or through parameters. If a task does not communicate with or affect the execution of any other task in the program in any way, it is said to be **disjoint**. Because tasks often work together to create simulations or solve problems and therefore are not disjoint, they must use some form of communication to either synchronize their executions or share data or both.

Synchronization is a mechanism that controls the order in which tasks execute. Two kinds of synchronization are required when tasks share data: cooperation and competition. **Cooperation synchronization** is required between task A and task B when task A must wait for task B to complete some specific activity before task A can begin or continue its execution. **Competition synchronization** is required between two tasks when both require the use of

some resource that cannot be simultaneously used. Specifically, if task A needs to access shared data location x while task B is accessing x , task A must wait for task B to complete its processing of x . So, for cooperation synchronization, tasks may need to wait for the completion of specific processing on which their correct operation depends, whereas for competition synchronization, tasks may need to wait for the completion of any other processing by any task currently occurring on specific shared data.

A simple form of cooperation synchronization can be illustrated by a common problem called the **producer-consumer problem**. This problem originated in the development of operating systems, in which one program unit produces some data value or resource and another uses it. Produced data are usually placed in a storage buffer by the producing unit and removed from that buffer by the consuming unit. The sequence of stores to and removals from the buffer must be synchronized. The consumer unit must not be allowed to take data from the buffer if the buffer is empty. Likewise, the producer unit cannot be allowed to place new data in the buffer if the buffer is full. This is a problem of cooperation synchronization because the users of the shared data structure must cooperate if the buffer is to be used correctly.

Competition synchronization prevents two tasks from accessing a shared data structure at exactly the same time—a situation that could destroy the integrity of that shared data. To provide competition synchronization, mutually exclusive access to the shared data must be guaranteed.

To clarify the competition problem, consider the following scenario: Suppose task A has the statement `TOTAL += 1`, where `TOTAL` is a shared integer variable. Furthermore, suppose task B has the statement `TOTAL *= 2`. Task A and task B could try to change `TOTAL` at the same time.

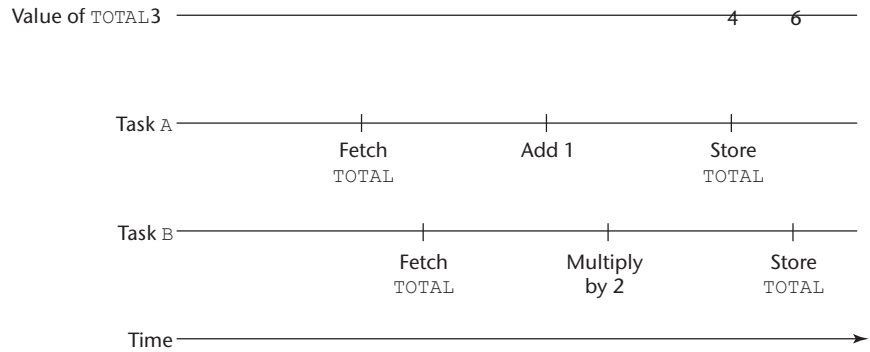
At the machine language level, each task may accomplish its operation on `TOTAL` with the following three-step process:

1. Fetch the value of `TOTAL`.
2. Perform the arithmetic operation.
3. Put the new value back in `TOTAL`.

Without competition synchronization, given the previously described operations performed by tasks A and B on `TOTAL`, four different values could result, depending on the order of the steps of the operations. Assume `TOTAL` has the value 3 before either A or B attempts to modify it. If task A completes its operation before task B begins, the value will be 8, which is assumed here to be correct. But if both A and B fetch the value of `TOTAL` before either task puts its new value back, the result will be incorrect. If A puts its value back first, the value of `TOTAL` will be 6. This case is shown in Figure 13.1. If B puts its value back first, the value of `TOTAL` will be 4. Finally, if B completes its operation before task A begins, the value will be 7. A situation that leads to these problems is sometimes called a **race condition**, because two or more tasks are racing to use the shared resource and the behavior of the program depends on which task arrives first (and wins the race). The importance of competition synchronization should now be clear.

Figure 13.1

The need for
competition
synchronization



One general method for providing mutually exclusive access (to support competition synchronization) to a shared resource is to consider the resource to be something that a task can possess and allow only a single task to possess it at a time. To gain possession of a shared resource, a task must request it. Possession will be granted only when no other task has possession. While a task possesses a resource, all other tasks are prevented from having access to that resource. When a task is finished with a shared resource that it possesses, it must relinquish that resource so it can be made available to other tasks.

Three methods of providing for mutually exclusive access to a shared resource are semaphores, which are discussed in Section 13.3; monitors, which are discussed in Section 13.4; and message passing, which is discussed in Section 13.5.

Mechanisms for synchronization must be able to delay task execution. Synchronization imposes an order of execution on tasks that is enforced with these delays. To understand what happens to tasks through their lifetimes, we must consider how task execution is controlled. Regardless of whether a machine has a single processor or more than one, there is always the possibility of there being more tasks than there are processors. A run-time system program called a **scheduler** manages the sharing of processors among the tasks. If there were never any interruptions and tasks all had the same priority, the scheduler could simply give each task a time slice, such as 0.1 second, and when a task's turn came, the scheduler could let it execute on a processor for that amount of time. Of course, there are several events that complicate this, for example, task delays for synchronization and for input or output operations. Because input and output operations are very slow relative to the processor's speed, a task is not allowed to keep a processor while it waits for completion of such an operation.

Tasks can be in several different states:

1. *New*: A task is in the new state when it has been created but has not yet begun its execution.
2. *Ready*: A ready task is ready to run but is not currently running. Either it has not been given processor time by the scheduler, or it had run previously but was blocked in one of the ways described in Paragraph 4

of this subsection. Tasks that are ready to run are stored in a queue that is often called the **task-ready queue**.

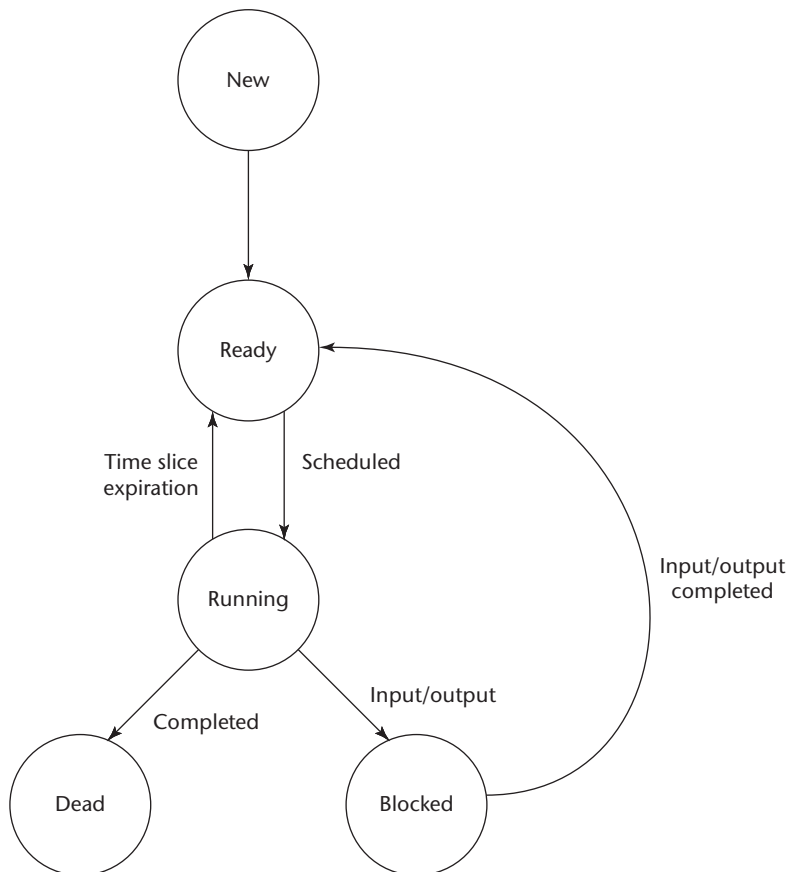
3. *Running*: A running task is one that is currently executing; that is, it has a processor and its code is being executed.
4. *Blocked*: A task that is blocked has been running, but that execution was interrupted by one of several different events, the most common of which is an input or output operation. In addition to input and output, some languages provide operations for the user program to specify that a task be blocked.
5. *Dead*: A dead task is no longer active in any sense. A task dies when its execution is completed or it is explicitly killed by the program.

A flow diagram of the states of a task is shown in Figure 13.2.

One important issue in task execution is the following: How is a ready task chosen to move to the running state when the task currently running has become blocked or whose time slice has expired? Several different algorithms

Figure 13.2

Flow diagram of task states



have been used for making this choice, some based on specifiable priority levels. The algorithm that does the choosing is implemented in the scheduler.

Associated with the concurrent execution of tasks and the use of shared resources is the concept of liveness. In the environment of sequential programs, a program has the **liveness** characteristic if it continues to execute, eventually leading to completion. In more general terms, liveness means that if some event—say, program completion—is supposed to occur, it will occur, eventually. That is, progress is continually made. In a concurrent environment and with shared resources, the liveness of a task can cease to exist, meaning that the program cannot continue and thus will never terminate.

For example, suppose task A and task B both need the shared resources X and Y to complete their work. Furthermore, suppose that task A gains possession of X and task B gains possession of Y. After some execution, task A needs resource Y to continue, so it requests Y but must wait until B releases it. Likewise, task B requests X but must wait until A releases it. Neither relinquishes the resource it possesses, and as a result, both lose their liveness, guaranteeing that execution of the program will never complete normally. This particular kind of loss of liveness is called **deadlock**. Deadlock is a serious threat to the reliability of a program, and therefore its avoidance demands serious consideration in both language and program design.

We are now ready to discuss some of the linguistic mechanisms for providing concurrent unit control.

13.2.2 Language Design for Concurrency

In some cases, concurrency is implemented through libraries. Among these is OpenMP, an applications programming interface to support shared memory multiprocessor programming in C, C++, and Fortran on a variety of platforms. Our interest in this book, of course, is language support for concurrency. A number of languages have been designed to support concurrency, beginning with PL/I in the mid-1960s and including the contemporary languages Ada 95, Java, C#, F#, Python, and Ruby.¹

13.2.3 Design Issues

The most important design issues for language support for concurrency have already been discussed at length: competition and cooperation synchronization. In addition to these, there are several design issues of secondary importance. Prominent among them is how an application can influence task scheduling. Also, there are the issues of how and when tasks start and end their executions, and how and when they are created.

1. In the cases of Python and Ruby, programs are interpreted, so there only can be logical concurrency. Even if the machine has multiple processors, these programs cannot make use of more than one.

Keep in mind that our discussion of concurrency is intentionally incomplete, and only the most important of the language design issues related to support for concurrency are discussed.

The following sections discuss three alternative approaches to the design issues for concurrency: semaphores, monitors, and message passing.

13.3 Semaphores

A semaphore is a simple mechanism that can be used to provide synchronization of tasks. Although semaphores are an early approach to providing synchronization, they are still used, both in contemporary languages and in library-based concurrency support systems. In the following paragraphs, we describe semaphores and discuss how they can be used for this purpose.

13.3.1 Introduction

In an effort to provide competition synchronization through mutually exclusive access to shared data structures, Edsger Dijkstra devised semaphores in 1965 (Dijkstra, 1968b). Semaphores can also be used to provide cooperation synchronization.

To provide limited access to a data structure, guards can be placed around the code that accesses the structure. A **guard** is a linguistic device that allows the guarded code to be executed only when a specified condition is true. So, a guard can be used to allow only one task to access a particular shared data structure at a time. A semaphore is an implementation of a guard. Specifically, a **semaphore** is a data structure that consists of an integer and a queue that stores task descriptors. A **task descriptor** is a data structure that stores all of the relevant information about the execution state of a task.

An integral part of a guard mechanism is a procedure for ensuring that all attempted executions of the guarded code eventually take place. The typical approach is to have requests for access that occur when access cannot be granted or stored in the task descriptor queue, from which they are later allowed to leave and execute the guarded code. This is the reason a semaphore must have both a counter and a task descriptor queue.

The only two operations provided for semaphores were originally named P and V by Dijkstra, after the two Dutch words *passeren* (to pass) and *vrygeren* (to release) (Andrews and Schneider, 1983). We will refer to these as *wait* and *release*, respectively, in the remainder of this section.

13.3.2 Cooperation Synchronization

Through much of this chapter, we use the example of a shared buffer used by producers and consumers to illustrate the different approaches to providing cooperation and competition synchronization. For cooperation

synchronization, such a buffer must have some way of recording both the number of empty positions and the number of filled positions in the buffer (to prevent buffer underflow and overflow). The counter component of a semaphore can be used for this purpose. One semaphore variable—for example, `emptyspots`—can use its counter to maintain the number of empty locations in a shared buffer used by producers and consumers, and another—say, `fullspots`—can use its counter to maintain the number of filled locations in the buffer. The queues of these semaphores can store the descriptors of tasks that have been forced to wait for access to the buffer. The queue of `emptyspots` can store producer tasks that are waiting for available positions in the buffer; the queue of `fullspots` can store consumer tasks waiting for values to be placed in the buffer.

Our example buffer is designed as an abstract data type in which all data enters the buffer through the subprogram `DEPOSIT`, and all data leaves the buffer through the subprogram `FETCH`. The `DEPOSIT` subprogram needs only to check with the `emptyspots` semaphore to see whether there are any empty positions. If there is at least one, it can proceed with the `DEPOSIT`, which must have the side effect of decrementing the counter of `emptyspots`. If the buffer is full, the caller to `DEPOSIT` must be made to wait in the `emptyspots` queue for an empty spot to become available. When the `DEPOSIT` is complete, the `DEPOSIT` subprogram increments the counter of the `fullspots` semaphore to indicate that there is one more filled location in the buffer.

The `FETCH` subprogram has the opposite sequence of `DEPOSIT`. It checks the `fullspots` semaphore to see whether the buffer contains at least one item. If it does, an item is removed and the `emptyspots` semaphore has its counter incremented by 1. If the buffer is empty, the calling task is put in the `fullspots` queue to wait until an item appears. When `FETCH` is finished, it must increment the counter of `emptyspots`.

The operations on semaphore types often are not direct—they are done through `wait` and `release` subprograms. Therefore, the `DEPOSIT` operation just described is actually accomplished in part by calls to `wait` and `release`. Note that `wait` and `release` must be able to access the task-ready queue.

The `wait` semaphore subprogram is used to test the counter of a given semaphore variable. If the value is greater than zero, the caller can carry out its operation. In this case, the counter value of the semaphore variable is decremented to indicate that there is now one fewer of whatever it counts. If the value of the counter is zero, the caller must be placed on the waiting queue of the semaphore variable, and the processor must be given to some other ready task.

The `release` semaphore subprogram is used by a task to allow some other task to have one of whatever the counter of the specified semaphore variable counts. If the queue of the specified semaphore variable is empty, which means no task is waiting, `release` increments its counter (to indicate there is one more of whatever is being controlled that is now available). If one or more tasks are waiting, `release` moves one of them from the semaphore queue to the ready queue.

The following are concise pseudocode descriptions of `wait` and `release`:

```
wait(aSemaphore)
if aSemaphore's counter > 0 then
    decrement aSemaphore's counter
else
    put the caller in aSemaphore's queue
    attempt to transfer control to some ready task
    (if the task-ready queue is empty, deadlock occurs)
end if

release(aSemaphore)
if aSemaphore's queue is empty (no task is waiting) then
    increment aSemaphore's counter
else
    put the calling task in the task-ready queue
    transfer control to a task from aSemaphore's queue
end
```

We can now present an example program that implements cooperation synchronization for a shared buffer. In this case, the shared buffer stores integer values and is a logically circular structure. It is designed for use by possibly multiple producer and consumer tasks.

The following pseudocode shows the definition of the producer and consumer tasks. Two semaphores are used to ensure against buffer underflow or overflow, thus providing cooperation synchronization. Assume that the buffer has length `BUFLen`, and the routines that actually manipulate it already exist as `FETCH` and `DEPOSIT`. Accesses to the counter of a semaphore are specified by dot notation. For example, if `fullspots` is a semaphore, its counter is referenced by `fullspots.count`.

```
semaphore fullspots, emptyspots;
fullspots.count = 0;
emptyspots.count = BUFLen;
task producer;
    loop
        -- produce VALUE --
        wait(emptyspots);    { wait for a space }
        DEPOSIT(VALUE);
        release(fullspots);  { increase filled spaces }
    end loop;
end producer;

task consumer;
    loop
        wait(fullspots);    { make sure it is not empty }
```



```

    FETCH(VALUE);
    release(emptyspots); { increase empty spaces }
    -- consume VALUE --
  end loop
end consumer;

```

The semaphore `fullspots` causes the `consumer` task to be queued to wait for a buffer entry if it is currently empty. The semaphore `emptyspots` causes the `producer` task to be queued to wait for an empty space in the buffer if it is currently full.

13.3.3 Competition Synchronization

Our buffer example does not provide competition synchronization. Access to the structure can be controlled with an additional semaphore. This semaphore need not count anything but can simply indicate with its counter whether the buffer is currently being used. The `wait` statement allows the access only if the semaphore's counter has the value 1, which indicates that the shared buffer is not currently being accessed. If the semaphore's counter has a value of 0, there is a current access taking place, and the task is placed in the queue of the semaphore. Notice that the semaphore's counter must be initialized to 1. The queues of semaphores must always be initialized to empty before use of the queue can begin.

A semaphore that requires only a binary-valued counter, like the one used to provide competition synchronization in the following example, is called a **binary semaphore**.

The example pseudocode that follows illustrates the use of semaphores to provide both competition and cooperation synchronization for a concurrently accessed shared buffer. The `access` semaphore is used to ensure mutually exclusive access to the buffer. Remember that there may be more than one producer and more than one consumer.

```

semaphore access, fullspots, emptyspots;
access.count = 1;
fullspots.count = 0;
emptyspots.count = BUFLen;

task producer;
  loop
    -- produce VALUE --
    wait(emptyspots);      { wait for a space }
    wait(access);          { wait for access }
    DEPOSIT(VALUE);
    release(access);       { relinquish access }
    release(fullspots);    { increase filled spaces }
  end loop;
end producer;

```

```

task consumer;
  loop
    wait(fullspots);      { make sure it is not empty }
    wait(access);        { wait for access }
    FETCH(VALUE);
    release(access);      { relinquish access }
    release(emptyspots); { increase empty spaces }
    -- consume VALUE --
  end loop
end consumer;

```

A brief look at this example may lead one to believe that there is a problem. Specifically, suppose that while a task is waiting at the `wait(access)` call in `consumer`, another task takes the last value from the shared buffer. Fortunately, this cannot happen, because the `wait(fullspots)` reserves a value in the buffer for the task that calls it by decrementing the `fullspots` counter.

There is one crucial aspect of semaphores that thus far has not been discussed. Recall the earlier description of the problem of competition synchronization: Operations on shared data must not overlap. If a second operation begins while an earlier operation is still in progress, the shared data can become corrupted. A semaphore is itself a shared data object, so the operations on semaphores are also susceptible to the same problem. It is therefore essential that semaphore operations be uninterruptible. Many computers have uninterruptible instructions that were designed specifically for semaphore operations. If such instructions are not available, then using semaphores to provide competition synchronization is a serious problem with no simple solution.

13.3.4 Evaluation

Using semaphores to provide cooperation synchronization creates an unsafe programming environment. There is no way to check statically for the correctness of their use, which depends on the semantics of the program in which they appear. In the buffer example, leaving out the `wait(emptyspots)` statement of the `producer` task would result in buffer overflow. Leaving out the `wait(fullspots)` statement of the `consumer` task would result in buffer underflow. Leaving out either of the releases would result in deadlock. These are cooperation synchronization failures.

The reliability problems that semaphores cause in providing cooperation synchronization also arise when using them for competition synchronization. Leaving out the `wait(access)` statement in either task can cause insecure access to the buffer. Leaving out the `release(access)` statement in either task results in deadlock. These are competition synchronization failures. Noting the danger in using semaphores, Per Brinch Hansen (1973) wrote, “The semaphore is an elegant synchronization tool for an ideal programmer who never makes mistakes.” Unfortunately, ideal programmers are rare.

13.4 Monitors

One solution to some of the problems of semaphores in a concurrent environment is to encapsulate shared data structures with their operations and hide their representations—that is, to make shared data structures abstract data types with some special restrictions. This solution can provide competition synchronization without semaphores by transferring responsibility for synchronization to the run-time system.

13.4.1 Introduction

When the concepts of data abstraction were being formulated, the people involved in that effort applied the same concepts to shared data in concurrent programming environments to produce monitors. According to Per Brinch Hansen (Brinch Hansen, 1977, p. xvi), Edsger Dijkstra suggested in 1971 that all synchronization operations on shared data be gathered into a single program unit. Brinch Hansen (1973) formalized this concept in the environment of operating systems. The following year, Hoare (1974) named these structures *monitors*.

The first programming language to incorporate monitors was Concurrent Pascal (Brinch Hansen, 1975). Modula (Wirth, 1977), CSP/k (Holt et al., 1978), and Mesa (Mitchell et al., 1979) also provide monitors. Among contemporary languages, monitors are supported by Ada, Java, and C#, all of which are discussed later in this chapter.

13.4.2 Competition Synchronization

One of the most important features of monitors is that shared data is resident in the monitor rather than in any of the client units. The programmer does not synchronize mutually exclusive access to shared data through the use of semaphores or other mechanisms. Because the access mechanisms are part of the monitor, implementation of a monitor can be made to guarantee synchronized access by allowing only one access at a time. Calls to monitor procedures are implicitly blocked and stored in a queue if the monitor is busy at the time of the call.

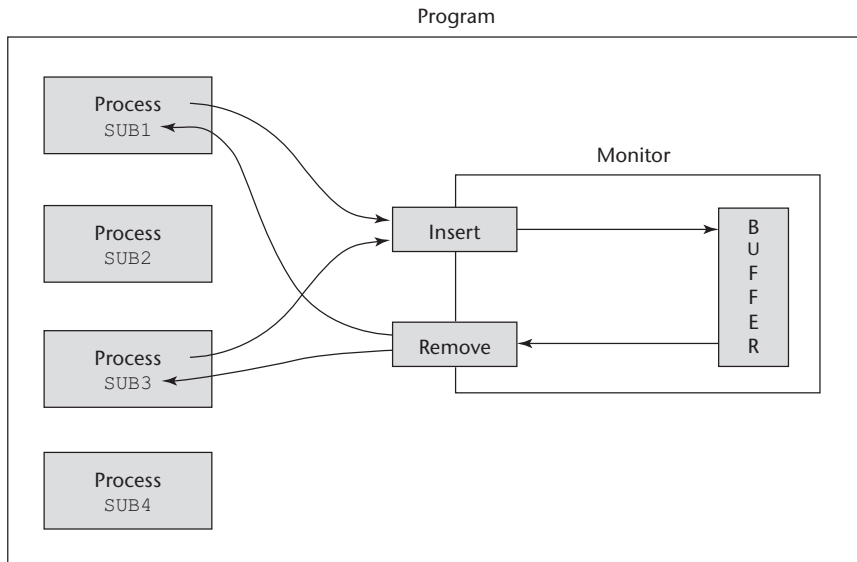
13.4.3 Cooperation Synchronization

Although mutually exclusive access to shared data is intrinsic with a monitor, cooperation between processes is still the task of the programmer. In particular, the programmer must guarantee that a shared buffer does not experience underflow or overflow. Different languages provide different ways of programming cooperation synchronization, all of which are related to semaphores.

A figure depicting a program containing four tasks and a monitor that provides synchronized access to a concurrently shared buffer is shown in Figure 13.3.

Figure 13.3

A program using a monitor to control access to a shared buffer



In this figure, the interface to the monitor is shown as the two boxes labeled `insert` and `remove` (for the insertion and removal of data). The monitor appears exactly like an abstract data type—a data structure with limited access—which is what a monitor is.

13.4.4 Evaluation

Monitors are a better way to provide competition synchronization than are semaphores, primarily because of the problems of semaphores, as discussed in Section 13.3. The cooperation synchronization is still a problem with monitors, as will be clear when Ada and Java implementations of monitors are discussed in the following sections.

Semaphores and monitors are equally powerful at expressing concurrency control—semaphores can be used to implement monitors and monitors can be used to implement semaphores.

Ada provides two ways to implement monitors. Ada 83 includes a general tasking model that can be used to support monitors. Ada 95 added a cleaner and more efficient way of constructing monitors, called *protected objects*. Both of these approaches use message passing as a basic model for supporting concurrency. The message-passing model allows concurrent units to be distributed, which monitors do not allow. Message passing is described in Section 13.5; Ada support for message passing is discussed in Section 13.6.

In Java, a monitor can be implemented in a class designed as an abstract data type, with the shared data being the type. Accesses to objects of the class are controlled by adding the **synchronized** modifier to the access methods. An example of a monitor for the shared buffer written in Java is given in Section 13.7.4.

C# has a predefined class, `Monitor`, which is designed for implementing monitors.

13.5 Message Passing

This section introduces the fundamental concept of message passing in concurrency. Note that this concept of message passing is unrelated to the message passing used in object-oriented programming to enact methods.

13.5.1 Introduction

The first efforts to design languages that provide the capability for message passing among concurrent tasks were those of Brinch Hansen (1978) and Hoare (1978). These pioneer developers of message passing also developed a technique for handling the problem of what to do when multiple simultaneous requests were made by other tasks to communicate with a given task. It was decided that some form of nondeterminism was required to provide fairness in choosing which among those requests would be taken first. This fairness can be defined in various ways, but in general, it means that all requesters are provided an equal chance of communicating with a given task (assuming that every requester has the same priority). Nondeterministic constructs for statement-level control, called *guarded commands*, were introduced by Dijkstra (1975). Guarded commands are discussed in Chapter 8. Guarded commands are the basis of the construct designed for controlling message passing.

13.5.2 The Concept of Synchronous Message Passing

Message passing can be either synchronous or asynchronous. Here, we describe synchronous message passing. The basic concept of synchronous message passing is that tasks are often busy, and when busy, they cannot be interrupted by other units. Suppose task A and task B are both in execution, and A wishes to send a message to B. Clearly, if B is busy, it is not desirable to allow another task to interrupt it. That would disrupt B's current processing. Furthermore, messages usually cause associated processing in the receiver, which might not be sensible if other processing is incomplete. The alternative is to provide a linguistic mechanism that allows a task to specify to other tasks when it is ready to receive messages. This approach is somewhat like an executive who instructs his or her secretary to hold all incoming calls until another activity, perhaps an important conversation, is completed. Later, when the current conversation is complete, the executive tells the secretary that he or she is now willing to talk to one of the callers who has been placed on hold.

A task can be designed so that it can suspend its execution at some point, either because it is idle or because it needs information from another unit before it can continue. This is like a person who is waiting for an important call. In some cases, there is nothing else to do but sit and wait. However, if task A is

waiting for a message at the time task B sends that message, the message can be transmitted. This actual transmission of the message is called a **rendezvous**. Note that a rendezvous can occur only if both the sender and receiver want it to happen. During a rendezvous, the information of the message can be transmitted in either or both directions.

Both cooperation and competition synchronization of tasks can be conveniently handled with the message-passing model, as described in the following section.

13.6 Ada Support for Concurrency

This section describes the support for concurrency provided by Ada. Ada 83 supports only synchronous message passing.

13.6.1 Fundamentals

The Ada design for tasks is partially based on the work of Brinch Hansen and Hoare in that message passing is the design basis and nondeterminism is used to choose among the tasks that have sent messages.

The full Ada tasking model is complex, and the following discussion of it is limited. The focus here will be on the Ada version of the synchronous message-passing mechanism.

Ada tasks can be more active than monitors. Monitors are passive entities that provide management services for the shared data they store. They provide their services, though only when those services are requested. When used to manage shared data, Ada tasks can be thought of as managers that can reside with the resource they manage. They have several mechanisms, some deterministic and some nondeterministic, that allow them to choose among competing requests for access to their resources.

There are two syntactic parts to an Ada task—a specification part and a body part—both with the same name. The interface of a task is its entry points or locations where it can accept messages from other tasks. Because these entry points are part of its interface, it is natural that they be listed in the specification part of a task. Because a rendezvous can involve an exchange of information, messages can have parameters; therefore, task entry points must also allow parameters, which must also be described in the specification part. In appearance, a task specification is similar to the package specification for an abstract data type.

As an example of an Ada task specification, consider the following code, which includes a single entry point named `Entry_1`, which has an in-mode parameter:

```
task Task_Example is  
  entry Entry_1(Item : in Integer);  
end Task_Example;
```

A task body must include some syntactic form of the entry points that correspond to the **entry** clauses in that task's specification part. In Ada, these task body entry points are specified by clauses that are introduced by the **accept** reserved word. An **accept clause** is defined as the range of statements beginning with the **accept** reserved word and ending with the matching **end** reserved word. **accept** clauses are themselves relatively simple, but other constructs in which they can be embedded can make their semantics complex. A simple **accept** clause has the following form:

```
accept entry_name (formal parameters) do
    ...
end entry_name;
```

The **accept** entry name matches the name in an **entry** clause in the associated task specification part. The optional parameters provide the means of communicating data between the caller and the called task. The statements between the **do** and the **end** define the operations that take place during the rendezvous. These statements are together called the **accept clause body**. During the actual rendezvous, the sender task is suspended.

Whenever an **accept** clause receives a message that it is not willing to accept, for whatever reason, the sender task must be suspended until the **accept** clause in the receiver task is ready to accept the message. Of course, the **accept** clause must also remember the sender tasks that have sent messages that were not accepted. For this purpose, each **accept** clause in a task has a queue associated with it that stores a list of other tasks that have unsuccessfully attempted to communicate with it.

The following is the skeletal body of the task whose specification was given previously:

```
task body Task_Example is
    begin
    loop
        accept Entry_1 (Item : in Integer) do
            ...
        end Entry_1;
    end loop;
end Task_Example;
```

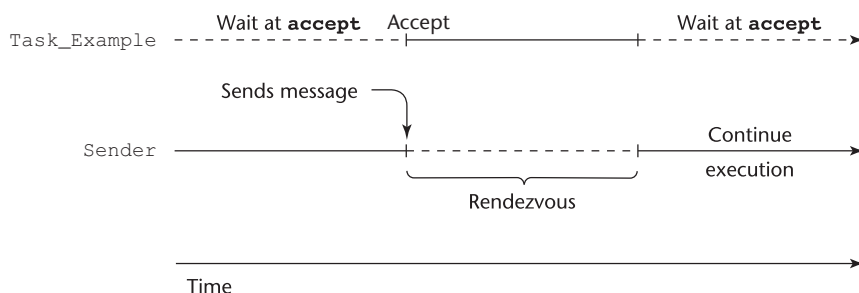
The **accept** clause of this task body is the implementation of the **entry** named `Entry_1` in the task specification. If the execution of `Task_Example` begins and reaches the `Entry_1` **accept** clause before any other task sends a message to `Entry_1`, `Task_Example` is suspended. If another task sends a message to `Entry_1` while `Task_Example` is suspended at its **accept**, a rendezvous occurs and the **accept** clause body is executed. Then, because of the loop, execution proceeds back to the **accept**. If no other task has sent a message to `Entry_1`, execution is again suspended to wait for the next message.

A rendezvous can occur in two basic ways in this simple example. First, the receiver task, `Task_Example`, can be waiting for another task to send a message to the `Entry_1` entry. When the message is sent, the rendezvous occurs. This is the situation described earlier. Second, the receiver task can be busy with one rendezvous, or with some other processing not associated with a rendezvous, when another task attempts to send a message to the same entry. In that case, the sender is suspended until the receiver is free to accept that message in a rendezvous. If several messages arrive while the receiver is busy, the senders are queued to wait their turn for a rendezvous.

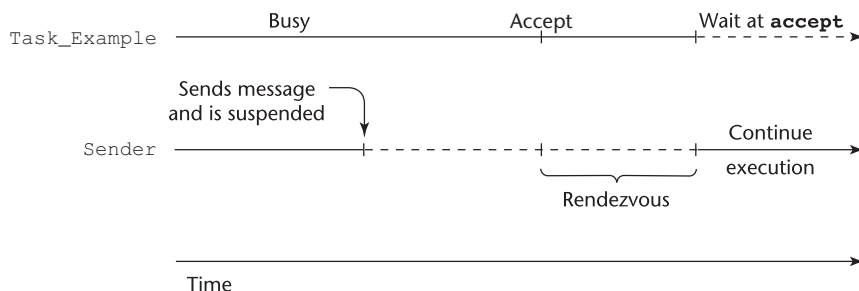
The two rendezvous just described are illustrated with the timeline diagrams in Figure 13.4.

Figure 13.4

Two ways a rendezvous with `Task_Example` can occur



(a) `Task_Example` waits for `Sender`



(b) `Sender` waits for `Task_Example`

Tasks need not have entry points. Such tasks are called **actor tasks** because they do not wait for a rendezvous in order to do their work. Actor tasks can rendezvous with other tasks by sending them messages. In contrast to actor tasks, a task can have **accept** clauses but not have any code outside those **accept** clauses, so it can only react to other tasks. Such a task is called a **server task**.

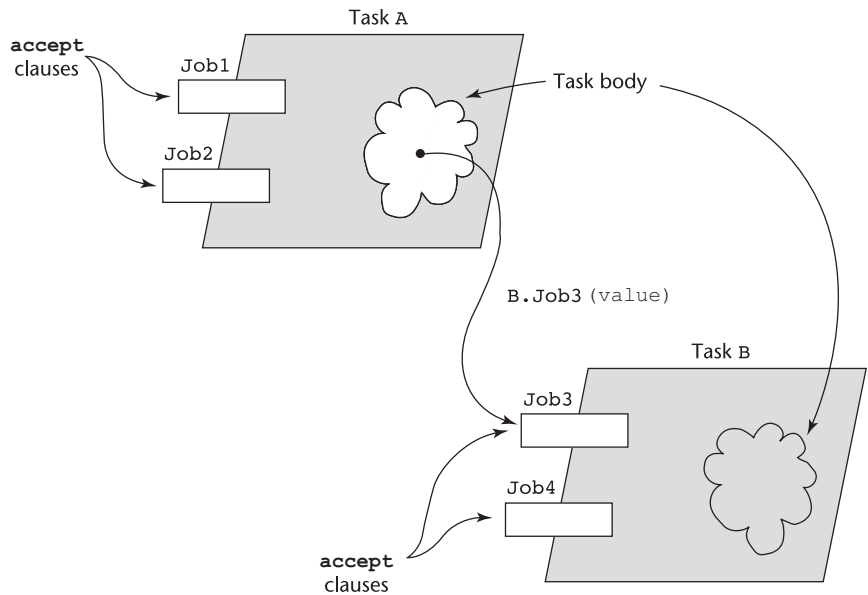
An Ada task that sends a message to another task must know the entry name in that task. However, the opposite is not true: A task entry need not

know the name of the task from which it will accept messages. This asymmetry is in contrast to the design of the language known as CSP, or Communicating Sequential Processes (Hoare, 1978). In CSP, which also uses the message-passing model of concurrency, tasks accept messages only from explicitly named tasks. The disadvantage of this is that libraries of tasks cannot be built for general use.

The usual graphical method of describing a rendezvous in which task A sends a message to task B is shown in Figure 13.5.

Figure 13.5

Graphical representation of a rendezvous caused by a message sent from task A to task B



Tasks are declared in the declaration part of a package, subprogram, or block. Statically created tasks² begin executing at the same time as the statements in the code to which that declarative part is attached. For example, a task declared in a main program begins execution at the same time as the first statement in the code body of the main program. Task termination, which is a complex issue, is discussed later in this section.

Tasks may have any number of entries. The order in which the associated **accept** clauses appear in the task dictates the order in which messages can be accepted. If a task has more than one entry point and requires them to be able to receive messages in any order, the task uses a **select** statement to enclose the entries. For example, suppose a task models the activities of a bank teller, who must serve customers at a walk-up station inside the bank and also serve

2. Tasks can also be dynamically created, but such tasks are not covered here.

customers at a drive-up window. The following skeletal teller task illustrates a **select** construct:

```
task body Teller is
begin
  loop
    select
      accept Drive_Up (formal parameters) do
        ...
      end Drive_Up;
      ...
    or
      accept Walk_Up (formal parameters) do
        ...
      end Walk_Up;
      ...
    end select;
  end loop;
end Teller;
```

In this task, there are two **accept** clauses, **Walk_Up** and **Drive_Up**, each of which has an associated queue. The action of the **select**, when it is executed, is to examine the queues associated with the two **accept** clauses. If one of the queues is empty, but the other contains at least one waiting message (customer), the **accept** clause associated with the waiting message or messages has a rendezvous with the task that sent the first message that was received. If both **accept** clauses have empty queues, the **select** waits until one of the entries is called. If both **accept** clauses have nonempty queues, one of the **accept** clauses is nondeterministically chosen to have a rendezvous with one of its callers. The loop forces the **select** statement to be executed repeatedly, forever.

The **end** of the **accept** clause marks the end of the code that assigns or references the formal parameters of the **accept** clause. The code, if there is any, between an **accept** clause and the next **or** (or the **end select**, if the **accept** clause is the last one in the **select**) is called the **extended accept clause**. The extended **accept** clause is executed only after the associated (immediately preceding) **accept** clause is executed. This execution of the extended **accept** clause is not part of the rendezvous and can take place concurrently with the execution of the calling task. The sender is suspended during the rendezvous, but it is put back in the ready queue when the end of the **accept** clause is reached. If an **accept** clause has no formal parameters, the **do-end** is not required, and the **accept** clause can consist entirely of an extended **accept** clause. Such an **accept** clause would be used exclusively for synchronization. Extended **accept** clauses are illustrated in the **Buf_Task** task in Section 13.6.3.

13.6.2 Cooperation Synchronization

Each **accept** clause can have a guard attached, in the form of a **when** clause, that can delay rendezvous. For example,

```
when not Full(Buffer) =>  
  accept Deposit(New_Value) do  
    ...  
  end
```

An **accept** clause with a **when** clause is either open or closed. If the Boolean expression of the **when** clause is currently true, that **accept** clause is called **open**; if the Boolean expression is false, the **accept** clause is called **closed**. An **accept** clause that does not have a guard is always open. An open **accept** clause is available for rendezvous; a closed **accept** clause cannot rendezvous.

Suppose there are several guarded **accept** clauses in a **select** clause. Such a **select** clause is usually placed in an infinite loop. The loop causes the **select** clause to be executed repeatedly, with each **when** clause evaluated on each repetition. Each repetition causes a list of open **accept** clauses to be constructed. If exactly one of the open clauses has a nonempty queue, a message from that queue is taken and a rendezvous takes place. If more than one of the open **accept** clauses has nonempty queues, one queue is chosen non-deterministically, a message is taken from that queue, and a rendezvous takes place. If the queues of all open clauses are empty, the task waits for a message to arrive at one of those **accept** clauses, at which time a rendezvous will occur. If a **select** is executed and every **accept** clause is closed, a run-time exception or error results. This possibility can be avoided either by making sure one of the **when** clauses is always true or by adding an **else** clause in the **select**. An **else** clause can include any sequence of statements, except an **accept** clause.

A **select** clause may have a special statement, **terminate**, that is selected only when it is open and no other **accept** clause is open. A **terminate** clause, when selected, means that the task is finished with its job but is not yet terminated. Task termination is discussed later in this section.

13.6.3 Competition Synchronization

The features described so far provide for cooperation synchronization and communication among tasks. Next, we discuss how mutually exclusive access to shared data structures can be enforced in Ada.

If access to a data structure is to be controlled by a task, then mutually exclusive access can be achieved by declaring the data structure within a task. The semantics of task execution usually guarantees mutually exclusive access to the structure, because only one **accept** clause in the task can be active at a given time. The only exceptions to this occur when tasks are nested in procedures

or other tasks. For example, if a task that defines a shared data structure has a nested task, that nested task can also access the shared structure, which could destroy the integrity of the data. Thus, tasks that are meant to control access to a shared data structure should not define tasks.

The following is an example of an Ada task that implements a monitor for a buffer. The buffer behaves very much like the buffer in Section 13.3, in which synchronization is controlled with semaphores.

```

task Buf_Task is
    entry Deposit(Item : in Integer);
    entry Fetch(Item : out Integer);
end Buf_Task;

task body Buf_Task is
    Bufsize : constant Integer := 100;
    Buf      : array (1..Bufsize) of Integer;
    Filled   : Integer range 0..Bufsize := 0;
    Next_In,
    Next_Out : Integer range 1..Bufsize := 1;
begin
    loop
        select
            when Filled < Bufsize =>
                accept Deposit(Item : in Integer) do
                    Buf(Next_In) := Item;
                end Deposit;
                Next_In := (Next_In mod Bufsize) + 1;
                Filled := Filled + 1;
            or
                when Filled > 0 =>
                    accept Fetch(Item : out Integer) do
                        Item := Buf(Next_Out);
                    end Fetch;
                    Next_Out := (Next_Out mod Bufsize) + 1;
                    Filled := Filled - 1;
            end select;
        end loop;
end Buf_Task;

```

In this example, both **accept** clauses are extended. These extended clauses can be executed concurrently with the tasks that called the associated **accept** clauses.

The tasks for a producer and a consumer that could use Buf_Task have the following form:

```

task Producer;
task Consumer;
task body Producer is
    New_Value : Integer;
begin

```

```

loop
    -- produce New_Value --
    Buf_Task.Deposit(New_Value);
end loop;
end Producer;

task body Consumer is
    Stored_Value : Integer;
begin
    loop
        Buf_Task.Fetch(Stored_Value);
        -- consume Stored_Value --
    end loop;
end Consumer;

```

13.6.4 Protected Objects

As we have seen, access to shared data can be controlled by enclosing the data in a task and allowing access only through task entries, which implicitly provide competition synchronization. One problem with this method is that it is difficult to implement the rendezvous mechanism efficiently. Ada 95 protected objects provide an alternative method of providing competition synchronization that need not involve the rendezvous mechanism.

A protected object is not a task; it is more like a monitor, as described in Section 13.4. Protected objects can be accessed either by protected subprograms or by entries that are syntactically similar to the **accept** clauses in tasks.³ The protected subprograms can be either protected procedures, which provide mutually exclusive read-write access to the data of the protected object, or protected functions, which provide concurrent read-only access to that data. Entries differ from protected subprograms in that they can have guards.

Within the body of a protected procedure, the current instance of the enclosing protected unit is defined to be a variable; within the body of a protected function, the current instance of the enclosing protected unit is defined to be a constant, which allows concurrent read-only access.

Entry calls to a protected object provide synchronous communication with one or more tasks using the same protected object. These entry calls provide access similar to that provided to the data enclosed in a task.

The buffer problem that is solved with a task in the previous subsection can be more simply solved with a protected object. Note that this example does not include protected subprograms.

```

protected Buffer is
    entry Deposit(Item : in Integer);
    entry Fetch(Item : out Integer);

```

3. Entries in protected object bodies use the reserved word **entry**, rather than the **accept** used in task bodies.

```

private
  Bufsize : constant Integer := 100;
  Buf      : array (1..Bufsize) of Integer;
  Filled   : Integer range 0..Bufsize := 0;
  Next_In,
  Next_Out : Integer range 1..Bufsize := 1;
end Buffer;

protected body Buffer is
  entry Deposit(Item : in Integer)
    when Filled < Bufsize is
    begin
      Buf(Next_In) := Item;
      Next_In := (Next_In mod Bufsize) + 1;
      Filled := Filled + 1;
    end Deposit;
  entry Fetch(Item : out Integer) when Filled > 0 is
    begin Item := Buf(Next_Out);
      Next_Out := (Next_Out mod Bufsize) + 1;
      Filled := Filled - 1;
    end Fetch;
end Buffer;

```

13.6.5 Evaluation

Using the general message-passing model of concurrency to construct monitors is like using Ada packages to support abstract data types—both are tools that are more general than is necessary. Protected objects are a better way to provide synchronized access to shared data.

In the absence of distributed processors with independent memories, the choice between monitors and tasks with message passing as a means of implementing synchronized access to shared data in a concurrent environment is somewhat a matter of taste. However, in the case of Ada, protected objects are clearly better than tasks for supporting concurrent access to shared data. Not only is the code simpler; it is also much more efficient.

For distributed systems, message passing is a better model for concurrency, because it naturally supports the concept of separate processes executing in parallel on separate processors.

13.7 Java Threads

The concurrent units in Java are methods named `run`, whose code can be in concurrent execution with other such methods (of other objects) and with the `main` method. The process in which the `run` methods execute is called a **thread**. Java's threads are lightweight tasks, which means that they all run in the same address space. This is different from Ada tasks, which are heavyweight

threads (they run in their own address spaces).⁴ One important result of this difference is that threads require far less overhead than Ada's tasks.

There are two ways to define a class with a `run` method. One of these is to define a subclass of the predefined class `Thread` and override its `run` method. However, if the new subclass has a necessary natural parent, then defining it as a subclass of `Thread` obviously will not work. In these situations, we define a subclass that inherits from its natural parent and implements the `Runnable` interface. `Runnable` provides the `run` method protocol, so any class that implements `Runnable` must define `run`. An object of the class that implements `Runnable` is passed to the `Thread` constructor. So, this approach still requires a `Thread` object, as will be seen in the example in Section 13.7.5.

In Ada, tasks can be either actors or servers and tasks communicate with each other through **accept** clauses. Java `run` methods are all actors and there is no mechanism for them to communicate with each other, except for the `join` method (see Section 13.7.1) and through shared data.

Java threads is a complex topic—this section only provides an introduction to its simplest but most useful parts.

13.7.1 The Thread Class

The `Thread` class is not the natural parent of any other classes. It provides some services for its subclasses, but it is not related in any natural way to their computational purposes. `Thread` is the only class available for creating concurrent Java programs. As previously stated, Section 13.7.5 will briefly discuss the use of the `Runnable` interface.

The `Thread` class includes five constructors and a collection of methods and constants. The `run` method, which describes the actions of the thread, is always overridden by subclasses of `Thread`. The `start` method of `Thread` starts its thread as a concurrent unit by calling its `run` method.⁵ The call to `start` is unusual in that control returns immediately to the caller, which then continues its execution, in parallel with the newly started `run` method.

Following is a skeletal subclass of `Thread` and a code fragment that creates an object of the subclass and starts the `run` method's execution in the new thread:

```
class MyThread extends Thread {
    public void run() { . . . }
}
. . .
Thread myTh = new MyThread();
myTh.start();
```

4. Actually, although Ada tasks behave as if they were heavyweight tasks, in some cases, they are now implemented as threads. This is sometimes done using libraries, such as the IBM Rational Apex Native POSIX Threading Library.

5. Calling the `run` method directly does not always work, because initialization that is sometimes required is included in the `start` method.

When a Java application program begins execution, a new thread is created (in which the `main` method will run) and `main` is called. Therefore, all Java application programs run in threads.

When a program has multiple threads, a scheduler must determine which thread or threads will run at any given time. In many cases, there is only a single processor available, so only one thread actually runs at a time. It is difficult to give a precise description of how the Java scheduler works, because the different implementations (Solaris, Windows, and so on) do not necessarily schedule threads in exactly the same way. Typically, however, the scheduler gives equal-size time slices to each ready thread in round-robin fashion, assuming all of these threads have the same priority. Section 13.7.2 describes how different priorities can be given to different threads.

The `Thread` class provides several methods for controlling the execution of threads. The `yield` method, which takes no parameters, is a request from the running thread to surrender the processor voluntarily.⁶ The thread is put immediately in the task-ready queue, making it ready to run. The scheduler then chooses the highest-priority thread from the task-ready queue. If there are no other ready threads with priority higher than the one that just yielded the processor, it may also be the next thread to get the processor.

The `sleep` method has a single parameter, which is the integer number of milliseconds that the caller of `sleep` wants the thread to be blocked. After the specified number of milliseconds has passed, the thread will be put in the task-ready queue. Because there is no way to know how long a thread will be in the task-ready queue before it runs, the parameter to `sleep` is the minimum amount of time the thread will *not* be in execution. The `sleep` method can throw an `InterruptedException`, which must be handled in the method that calls `sleep`. Exceptions are described in detail in Chapter 14.

The `join` method is used to force a method to delay its execution until the `run` method of another thread has completed its execution. `join` is used when the processing of a method cannot continue until the work of the other thread is complete. For example, we might have the following `run` method:

```
public void run() {  
    . . .  
    Thread myTh = new Thread();  
    myTh.start();  
    // do part of the computation of this thread  
    myTh.join(); // Wait for myTh to complete  
    // do the rest of the computation of this thread  
}
```

The `join` method puts the thread that calls it in the blocked state, which can be ended only by the completion of the thread on which `join` was called. If that thread happens to be blocked, there is the possibility of deadlock. To prevent

6. The `yield` method is actually defined to be a “suggestion” to the scheduler, which it may or may not follow (though it usually does).

this, `join` can be called with a parameter, which is the time limit in milliseconds of how long the calling thread will wait for the called thread to complete. For example, the following call to `join` will cause the calling thread to wait two seconds for `myTh` to complete. If it has not completed its execution after two seconds have passed, the calling thread is put back in the ready queue, which means that it will continue its execution as soon as it is scheduled.

```
myTh.join(2000);
```

Early versions of Java included three more `Thread` methods: `stop`, `suspend`, and `resume`. All three of these have been deprecated because of safety problems. The `stop` method is sometimes overridden with a simple method that destroys the thread by setting its reference variable to `null`.

The normal way a `run` method ends its execution is by reaching the end of its code. However, in many cases, threads run until told to terminate. Regarding this, there is the question of how a thread can determine whether it should continue or end. The `interrupt` method is one way to communicate to a thread that it should stop. This method does not stop the thread; rather, it sends the thread a message that actually just sets a bit in the thread object, which can be checked by the thread. The bit is checked with the predicate method, `isInterrupted`. This is not a complete solution, because the thread one is attempting to interrupt may be sleeping or waiting at the time the `interrupt` method is called, which means that it will not be checking to see if it has been interrupted. For these situations, the `interrupt` method also throws an exception, `InterruptedException`, which also causes the thread to awaken (from sleeping or waiting). So, a thread can periodically check to see whether it has been interrupted and if so, whether it can terminate. The thread cannot miss the interrupt, because if it was asleep or waiting when the interrupt occurred, it will be awakened by the interrupt. Actually, there are more details to the actions and uses of `interrupt`, but they are not covered here (Arnold et al., 2006).

13.7.2 Priorities

The priorities of threads need not all be the same. A thread's default priority initially is the same as the thread that created it. If `main` creates a thread, its default priority is the constant `NORM_PRIORITY`, which is usually 5. `Thread` defines two other priority constants, `MAX_PRIORITY` and `MIN_PRIORITY`, whose values are usually 10 and 1, respectively.⁷ The priority of a thread can be changed with the method `setPriority`. The new priority can be any of the predefined constants or any other number between `MIN_PRIORITY` and `MAX_PRIORITY`. The `getPriority` method returns the current priority of a thread. The priority constants are defined in `Thread`.

7. The number of priorities is implementation dependent, so there may be fewer or more than 10 levels in some implementations.

When there are threads with different priorities, the scheduler's behavior is controlled by those priorities. When the executing thread is blocked or killed or the time slice for it expires, the scheduler chooses the thread from the task-ready queue that has the highest priority. A thread with lower priority will run only if one of higher priority is not in the task-ready queue when the opportunity arises.

13.7.3 Semaphores

The `java.util.concurrent.Semaphore` package defines the `Semaphore` class. Objects of this class implement counting semaphores. A counting semaphore has a counter, but no queue for storing thread descriptors. The `Semaphore` class defines two methods, `acquire` and `release`, which correspond to the `wait` and `release` operations described in Section 13.3.

The basic constructor for `Semaphore` takes one integer parameter, which initializes the semaphore's counter. For example, the following could be used to initialize the `fullspots` and `emptyspots` semaphores for the buffer example of Section 13.3.2:

```
fullspots = new Semaphore(0);  
emptyspots = new Semaphore(BUFLEN);
```

The deposit operation of the producer method would appear as follows:

```
emptyspots.acquire();  
deposit(value);  
fullspots.release();
```

Likewise, the fetch operation of the consumer method would appear as follows:

```
fullspots.acquire();  
fetch(value);  
emptyspots.release();
```

The `deposit` and `fetch` methods could use the approach used in Section 13.7.4 to provide the competition synchronization required for the accesses to the buffer.

13.7.4 Competition Synchronization

Java methods (but not constructors) can be specified to be **synchronized**. A synchronized method called through a specific object must complete its execution before any other synchronized method can run on that object. Competition synchronization on an object is implemented by specifying that the methods that access shared data are synchronized. The synchronized mechanism is implemented as follows: Every Java object has a lock. Synchronized methods must acquire the lock of the object before they are allowed to execute, which prevents other synchronized methods from executing on the object

during that time. A synchronized method releases the lock on the object on which it runs when it completes its execution, even if that completion is due to an exception. Consider the following skeletal class definition:

```
class ManageBuf {
    private int [100] buf;
    . . .
    public synchronized void deposit(int item) { . . . }
    public synchronized int fetch() { . . . }
    . . .
}
```

The two methods defined in `ManageBuf` are both defined to be **synchronized**, which prevents them from interfering with each other while executing on the same object, when they are called by separate threads.

An object whose methods are all synchronized is effectively a monitor. Note that an object may have one or more synchronized methods, as well as one or more unsynchronized methods. An unsynchronized method can run on an object at anytime, even during the execution of a synchronized method.

In some cases, the number of statements that deal with the shared data structure is significantly less than the number of other statements in the method in which it resides. In these cases, it is better to synchronize the code segment that changes the shared data structure rather than the whole method. This can be done with a so-called *synchronized statement*, whose general form is as follows:

```
synchronized (expression) {
    statements
}
```

The expression in this code must evaluate to an object and the statement can be a single statement or a compound statement. The object is locked during execution of the statement or compound statement, so the statement or compound statement is executed exactly as if it were the body of a synchronized method.

An object that has synchronized methods defined for it must have a queue associated with it that stores the synchronized methods that have attempted to execute on it while it was being operated upon by another synchronized method. Actually, every object has a queue called the **intrinsic condition queue**. These queues are implicitly supplied. When a synchronized method completes its execution on an object, a method that is waiting in the object's intrinsic condition queue, if there is such a method, is put in the task-ready queue.

13.7.5 Cooperation Synchronization

Cooperation synchronization in Java is implemented with the `wait`, `notify`, and `notifyAll` methods, all of which are defined in `Object`, the root class of all Java classes. All classes except `Object` inherit these methods. Every object

has a wait list of all of the threads that have called `wait` on the object. The `notify` method is called to tell one waiting thread that an event that it may have been waiting for has occurred. The specific thread that is awakened by `notify` cannot be determined, because the Java Virtual Machine (JVM) chooses one from the wait list of the thread object at random. Because of this, along with the fact that the waiting threads may be waiting for different conditions, the `notifyAll` method is often used, rather than `notify`. The `notifyAll` method awakens all of the threads on the object's wait list by putting them in the task-ready queue.

The methods `wait`, `notify`, and `notifyAll` can be called only from within a synchronized method, because they use the lock placed on an object by such a method. The call to `wait` is always put in a **while** loop that is controlled by the condition for which the method is waiting. The **while** loop is necessary because the `notify` or `notifyAll` that awakened the thread may have been called because of a change in a condition other than the one for which the thread was waiting. If it was a call to `notifyAll`, there is even a smaller chance that the condition being waited for is now true. Because of the use of `notifyAll`, some other thread may have changed the condition to false since it was last tested.

The `wait` method can throw `InterruptedException`, which is a descendant of `Exception`. Java's exception handling is discussed in Chapter 14. Therefore, any code that calls `wait` must also catch `InterruptedException`. Assuming the condition being waited for is called `theCondition`, the conventional way to use `wait` is as follows:

```
try {
    while (!theCondition)
        wait();
    -- Do whatever is needed after theCondition comes true
}
catch(InterruptedException myProblem) { . . . }
```

The following program implements a circular queue for storing `int` values. It illustrates both cooperation and competition synchronization.

```
// Queue
// This class implements a circular queue for storing int
// values. It includes a constructor for allocating and
// initializing the queue to a specified size. It has
// synchronized methods for inserting values into and
// removing values from the queue.

class Queue {
    private int [] que;
    private int nextIn,
               nextOut,
               filled,
               queSize;
```

```

public Queue(int size) {
    que = new int [size];
    filled = 0;
    nextIn = 1;
    nextOut = 1;
    queSize = size;
} /** end of Queue constructor

public synchronized void deposit (int item)
    throws InterruptedException {
    try {
        while (filled == queSize)
            wait();
        que [nextIn] = item;
        nextIn = (nextIn % queSize) + 1;
        filled++;
        notifyAll();
    } /** end of try clause
    catch(InterruptedException e) {}
} /** end of deposit method

public synchronized int fetch()
    throws InterruptedException {
    int item = 0;
    try {
        while (filled == 0)
            wait();
        item = que [nextOut];
        nextOut = (nextOut % queSize) + 1;
        filled--;
        notifyAll();
    } /** end of try clause
    catch(InterruptedException e) {}
    return item;
} /** end of fetch method
} /** end of Queue class

```

Notice that the exception handler (**catch**) does nothing here.

Classes to define producer and consumer objects that could use the `Queue` class can be defined as follows:

```

class Producer extends Thread {
    private Queue buffer;
    public Producer(Queue que) {
        buffer = que;
    }
    public void run() {
        int new_item;
        while (true) {
            //-- Create a new_item

```

```

        buffer.deposit(new_item);
    }
}

class Consumer extends Thread {
    private Queue buffer;
    public Consumer(Queue que) {
        buffer = que;
    }
    public void run() {
        int stored_item;
        while (true) {
            stored_item = buffer.fetch();
            //-- Consume the stored_item
        }
    }
}

```

The following code creates a `Queue` object, and a `Producer` and a `Consumer` object, both attached to the `Queue` object, and starts their execution:

```

Queue buff1 = new Queue(100);
Producer producer1 = new Producer(buff1);
Consumer consumer1 = new Consumer(buff1);
producer1.start();
consumer1.start();

```

We could define one or both of the `Producer` and the `Consumer` as implementations of the `Runnable` interface rather than as subclasses of `Thread`. The only difference is in the first line, which would now appear as follows:

```

class Producer implements Runnable { . . . }

```

To create and run an object of such a class, it is still necessary to create a `Thread` object that is connected to the object. This is illustrated in the following code:

```

Producer producer1 = new Producer(buff1);
Thread producerThread = new Thread(producer1);
producerThread.start();

```

Note that the buffer object is passed to the `Producer` constructor and the `Producer` object is passed to the `Thread` constructor.

13.7.6 Nonblocking Synchronization

Java includes some classes for controlling accesses to certain variables that do not include blocking or waiting. The `java.util.concurrent.atomic` package defines classes that allow certain nonblocking synchronized access to **int**, **long**, and **boolean** primitive type variables, as well as references and arrays. For example, the `AtomicInteger` class defines getter and setter methods, as well as methods for add, increment, and decrement operations. These operations are all atomic; that is, they cannot be interrupted, so locks are not required to guarantee the integrity of the values of the affected variables in a multithreaded program. This is fine-grained synchronization—just a single variable. Most machines now have atomic instructions for these operations on **int** and **long** types, so they are often easy to implement (implicit locks are not required).

The advantage of nonblocking synchronization is efficiency. A nonblocking access that does not occur during contention will be no slower, and usually faster than one that uses **synchronized**. A nonblocking access that occurs during contention definitely will be faster than one that uses **synchronized**, because the latter will require suspension and rescheduling of threads.

13.7.7 Explicit Locks

Java 5.0 introduced explicit locks as an alternative to **synchronized** method and blocks, which provide implicit locks. The `Lock` interface declares the `lock`, `unlock`, and `tryLock` methods. The predefined `ReentrantLock` class implements the `Lock` interface. To lock a block of code, the following idiom can be used:

```
Lock lock = new ReentrantLock();
. . .
Lock.lock();
try {
    // The code that accesses the shared data
} finally {
    Lock.unlock();
}
```

This skeletal code creates a `Lock` object and calls the `lock` method on the `Lock` object. Then, it uses a **try** block to enclose the critical code. The call to `unlock` is in a **finally** clause to guarantee the lock is released, regardless of what happens in the **try** block.

There are at least two situations in which explicit locks are used rather than implicit locks: First, if the application needs to try to acquire a lock but cannot wait forever for it, the `Lock` interface includes a method, `tryLock`, that takes a time limit parameter. If the lock is not acquired within the time limit, execution continues at the statement following the call to `tryLock`. Second,

explicit locks are used when it is not convenient to have the lock-unlock pairs block structured. Implicit locks are always unlocked at the end of the compound statement in which they are locked. Explicit locks can be unlocked anywhere in the code, regardless of the structure of the program.

One danger of using explicit locks (and is not the case with using implicit locks) is that of omitting the unlock. Implicit locks are implicitly unlocked at the end of the locked block. However, explicit locks stay locked until explicitly unlocked, which can potentially be never.

As stated previously, each object has an intrinsic condition queue, which stores threads waiting for a condition on the object. The `wait`, `notify`, and `notifyAll` methods are the API for an intrinsic condition queue. Because each object can have just one condition queue, a queue may have threads in it waiting for different conditions. For example, the queue for our buffer example `Queue` can have threads waiting for either of two conditions (`filled == queueSize` or `filled == 0`). That is the reason why the buffer uses `notifyAll`. (If it used `notify`, only one thread would be awakened, and it might be one that was waiting for a different condition than the one that actually became true.) However, `notifyAll` is expensive to use, because it awakens all threads waiting on an object and all must check their condition to determine which one runs. Furthermore, to check their condition, they must first acquire the lock on the object.

An alternative to using the intrinsic condition queue is the `Condition` interface, which uses a condition queue associated with a `Lock` object. It also declares alternatives to `wait`, `notify`, and `notifyAll` named `await`, `signal`, and `signalAll`. There can be any number of `Condition` objects with one `Lock` object. With `Condition`, `signal`, rather than `signalAll`, can be used, which is both easier to understand and more efficient, in part because it results in fewer context switches.

13.7.8 Evaluation

Java's support for concurrency is relatively simple but effective. All Java run methods are actor tasks and there is no mechanism for communication, except through shared data, as there is among Ada tasks. Because they are heavyweight threads, Ada's tasks easily can be distributed to different processors; in particular, different processors with different memories, which could be on different computers in different places. These kinds of systems are not possible with Java's threads.

13.8 C# Threads

Although C#'s threads are loosely based on those of Java, there are significant differences. Following is a brief overview of C#'s threads.

13.8.1 Basic Thread Operations

Rather than just methods named `run`, as in Java, any C# method can run in its own thread. When C# threads are created, they are associated with an instance of a predefined delegate, `ThreadStart`. When execution of a thread is started, its delegate has the address of the method it is supposed to run. So, execution of a thread is controlled through its associated delegate.

A C# thread is created by creating a `Thread` object. The `Thread` constructor must be sent an instantiation of `ThreadStart`, to which must be sent the name of the method that is to run in the thread. For example, we might have

```
public void MyRun1 () { . . . }  
. . .  
Thread myThread = new Thread(new ThreadStart(MyRun1));
```

In this example, we create a thread named `myThread`, whose delegate points to the method `MyRun1`. So, when the thread begins execution it calls the method whose address is in its delegate. In this example, `myThread` is the delegate and `MyRun1` is the method.

Unlike Java, in which all threads are actors, C# has two categories of threads: actors and servers. Actor threads are not called specifically; rather, they are started. Also, the methods that they execute do not take parameters or return values. As with Java, creating a thread does not start its concurrent execution. For actor threads, execution must be requested through a method of the `Thread` class, in this case named `Start`, as in

```
myThread.Start();
```

As in Java, a thread can be made to wait for another thread to finish its execution before continuing, using the similarly named method `Join`. For example, suppose thread A has the following call:

```
B.Join();
```

Thread A will be blocked until thread B exits.

The `Join` method can take an `int` parameter, which specifies a time limit in milliseconds that the caller will wait for the thread to finish.

A thread can be suspended for a specified amount of time with `Sleep`, which is a public static method of `Thread`. The parameter to `Sleep` is an integer number of milliseconds. Unlike its Java relative, C#'s `Sleep` does not raise any exceptions, so it need not be called in a `try` block.

A thread can be terminated with the `Abort` method, although it does not literally kill the thread. Instead, it throws `ThreadAbortException`, which the thread can catch. When the thread catches this exception, it usually deallocates any resources it allocated, and then ends (by getting to the end of its code).

A server thread runs only when called through its delegate. These threads are called servers because they provide some service when it is requested. Server threads are more interesting than actor threads because they usually interact with other threads and often must have their execution synchronized with other threads.

Recall from Chapter 9, that any C# method can be called indirectly through a delegate. Such calls can be made by treating the delegate object as if it were the name of the method. This was actually an abbreviation for a call to a delegate method named `Invoke`. So, if a delegate object's name is `chgfun1` and the method it references takes one **int** parameter, we could call that method with either of the following statements:

```
chgfun1(7);
chgfun1.Invoke(7);
```

These calls are synchronous; that is, when the method is called, the caller is blocked until the method completes its execution. C# also supports asynchronous calls to methods that execute in threads. When a thread is called asynchronously, the called thread and the caller thread execute concurrently, because the caller is not blocked during the execution of the called thread.

A thread is called asynchronously through the delegate instance method `BeginInvoke`, to which are sent the parameters for the method of the delegate, along with two additional parameters, one of type `AsyncCallback` and the other of type **object**. `BeginInvoke` returns an object that implements the `IAsyncResult` interface. The delegate class also defines the `EndInvoke` instance method, which takes one parameter of type `IAsyncResult` and returns the same type that is returned by the method encapsulated in the delegate object. To call a thread asynchronously, we call it with `BeginInvoke`. For now, we will use **null** for the last two parameters. Suppose we have the following method declaration and thread definition:

```
public float MyMethod1(int x);
. . .
Thread myThread = new Thread(new ThreadStart(MyMethod1));
```

The following statement calls `MyMethod` asynchronously:

```
IAsyncResult result = myThread.BeginInvoke(10, null, null);
```

The return value of the called thread is fetched with `EndInvoke` method, which takes as its parameter the object (of type `IAsyncResult`) returned by `BeginInvoke`. `EndInvoke` returns the return value of the called thread. For example, to get the **float** result of the call to `MyMethod`, we would use the following statement:

```
float returnValue = EndInvoke(result);
```

If the caller must continue some work while the called thread executes, it must have a way to determine when the called thread is finished. For this,

the `IAsyncResult` interface defines the `IsCompleted` property. While the called thread is executing, the caller can include code it can execute in a **while** loop that depends on `IsCompleted`. For example, we could have the following:

```
IAsyncResult result = myThread.BeginInvoke(10, null, null);
while(!result.IsCompleted) {
    // Do some computation
}
```

This is an effective way to accomplish something in the calling thread while waiting for the called thread to complete its work. However, if the amount of computation in the **while** loop is relatively small, this is an inefficient way to use that time (because of the time required to test `IsCompleted`). An alternative is to give the called thread a delegate with the address of a callback method and have it call that method when it is finished. The delegate is sent as the second last parameter to `BeginInvoke`. For example, consider the following call to `BeginInvoke`:

```
IAsyncResult result = myThread.BeginInvoke(10,
    new AsyncCallback(MyMethodComplete), null);
```

The callback method is defined in the caller. Such methods often simply set a Boolean variable, for example named `isDone`, to **true**. No matter how long the called thread takes, the callback method is called only once.

13.8.2 Synchronizing Threads

There are three different ways that C# threads can be synchronized: the `Interlocked` class, the `Monitor` class from the `System.Threading` namespace, and the `lock` statement. Each of these mechanisms is designed for a specific need. The `Interlocked` class is used when the only operations that need to be synchronized are the incrementing and decrementing of an integer. These operations are done atomically with the two methods of `Interlocked`, `Increment` and `Decrement`, both of which take a reference to an integer as the parameter. For example, to increment a shared integer named `counter` in a thread, we could use

```
Interlocked.Increment(ref counter);
```

The `lock` statement is used to mark a critical section of code in a thread. The syntax of this is as follows:

```
lock(token) {
    // The critical section
}
```

If the code to be synchronized is in a private instance method, the token is the current object, so `this` is used as the token for `lock`. If the code to be synchronized is in a public instance method, a new instance of `object` is created (in the class of the method with the code to be synchronized) and a reference to it is used as the token for `lock`.

The `Monitor` class defines five methods, `Enter`, `Wait`, `Pulse`, `PulseAll`, and `Exit`, which can be used to provide more control of the synchronization of threads. The `Enter` method, which takes an object reference as its parameter, marks the beginning of synchronization of the thread on that object. The `Wait` method suspends execution of the thread and instructs the Common Language Runtime (CLR) of .NET that this thread wants to resume its execution the next time there is an opportunity. The `Pulse` method, which also takes an object reference as its parameter, notifies one waiting thread that it now has a chance to run again. `PulseAll` is similar to Java's `notifyAll`. Threads that have been waiting are run in the order in which they called the `Wait` method. The `Exit` method ends the critical section of the thread.

The `lock` statement is compiled into a monitor, so `lock` is shorthand for a monitor. A monitor is used when the additional control (for example, with `Wait` and `PulseAll`) is needed.

.NET 4.0 added a collection of generic concurrent data structures, including structures for queues, stacks, and bags.⁸ These new classes are thread safe, meaning that they can be used in a multithreaded program without requiring the programmer to worry about competition synchronization. The `System.Collections.Concurrent` namespace defines these classes, whose names are `ConcurrentQueue<T>`, `ConcurrentStack<T>`, and `ConcurrentBag<T>`. So, our producer-consumer queue program could be written in C# using a `ConcurrentQueue<T>` for the data structure and there would be no need to program the competition synchronization for it. Because these concurrent collections are defined in .NET, they are also available in all of the other .NET languages.

13.8.3 Evaluation

C#'s threads are a slight improvement over those of its predecessor, Java. For one thing, any method can be run in its own thread. Recall that in Java, only methods named `run` can run in their own threads. Java supports actor threads only, but C# supports both actor and server threads. Thread termination is also cleaner with C# (calling a method (`Abort`) is more elegant than setting the thread's pointer to `null`). Synchronization of thread execution is more sophisticated in C#, because C# has several different mechanisms, each for a specific application. Java's `Lock` variables are similar to the locks of C#, except that in Java, a lock must be explicitly unlocked with a call to `unlock`. This provides one more way to create erroneous code. C# threads, like those of Java, are lightweight, so although they are more efficient, they cannot be as

8. Bags are unordered collections of objects.

versatile as Ada's tasks. The availability of the concurrent collection classes is another advantage C# has over the other nonfunctional languages discussed in this chapter.

13.9 Concurrency in Functional Languages

This section provides a brief overview of support for concurrency in several functional programming languages.

13.9.1 Multi-LISP

Multi-LISP (Halstead, 1985) is an extension to Scheme that allows the programmer to specify program parts that can be executed concurrently. These forms of concurrency are implicit; the programmer is simply telling the compiler (or interpreter) some parts of the program that can be run concurrently.

One of the ways a programmer can tell the system about possible concurrency is the `pcall` construct. If a function call is embedded in a `pcall` construct, the parameters to the function can be evaluated concurrently. For example, consider the following `pcall` construct:

```
(pcall f a b c d)
```

The function is `f`, with parameters `a`, `b`, `c`, and `d`. The effect of `pcall` is that the parameters of the function can be evaluated concurrently (any or all of the parameters could be complicated expressions). Unfortunately, whether this process can be safely used, that is, without affecting the semantics of the function evaluation, is the responsibility of the programmer. This is actually a simple matter if the language does not allow side effects or if the programmer designed the function not to have side effects or at least to have limited ones. However, Multi-LISP does allow some side effects. If the function was not written to avoid side effects, it may be difficult for the programmer to determine whether `pcall` can be safely used.

The `future` construct of Multi-LISP is a more interesting and potentially more productive source of concurrency. As with `pcall`, a function call is wrapped in a `future` construct. Such a function is evaluated in a separate thread, with the parent thread continuing its execution. The parent thread continues until it needs to use the return value of the function. If the function has not completed its execution when its result is needed, the parent thread waits until it has before it continues.

If a function has two or more parameters, they can also be wrapped in `future` constructs, in which case their evaluations can be done concurrently in separate threads.

These are the only additions to Scheme in Multi-LISP.

13.9.2 Concurrent ML

Concurrent ML (CML) is an extension to ML that includes a form of threads and a form of synchronous message passing to support concurrency. The language is completely described in Reppy (1999).

A thread is created in CML with the `spawn` primitive, which takes the function as its parameter. In many cases, the function is specified as an anonymous function. As soon as the thread is created, the function begins its execution in the new thread. The return value of the function is discarded. The effects of the function are either output produced or through communications with other threads. Either the parent thread (the one that spawned the new thread) or the child thread (the new one) could terminate first and it would not affect the execution of the other.

Channels provide the means of communicating between threads. A channel is created with the `channel` constructor. For example, the following statement creates a channel of arbitrary type named `mychannel`:

```
let val mychannel = channel()
```

The two primary operations (functions) on channels are for sending (`send`) and receiving (`recv`) messages. The type of the message is inferred from the send operation. For example, the following function call sends the integer value 7, and therefore the type of the channel is then inferred to be integer:

```
send(mychannel, 7)
```

The `recv` function names the channel as its parameter. Its return value is the value it received.

Because CML communications are synchronous, a message is both sent and received only if both the sender and the receiver are ready. If a thread sends a message on a channel and no other thread is ready to receive on that channel, the sender is blocked and waits for another thread to execute a `recv` on the channel. Likewise, if a `recv` is executed on a channel by a thread but no other thread has sent a message on that channel, the thread that ran the `recv` is blocked and waits for a message on that channel.

Because channels are types, functions can take them as parameters.

As was the case with Ada's synchronous message passing, an issue with CML synchronous message passing is deciding which message to choose when more than one channel has received one. And the same solution is used: the guarded command `do-od` construct that chooses randomly among messages to different channels.

The synchronization mechanism of CML is the *event*. An explanation of this complicated mechanism is beyond the scope of this chapter (and this book).

13.9.3 F#

Part of the F# support for concurrency is based on the same .NET classes that are used by C#, specifically `System.Threading.Thread`. For example, suppose we want to run the function `myConMethod` in its own thread. The following function, when called, will create the thread and start the execution of the function in the new thread:

```
let createThread() =
    let newThread = new Thread(myConMethod)
    newThread.Start()
```

Recall that in C#, it is necessary to create an instance of a predefined delegate, `ThreadStart`, send its constructor the name of the subprogram, and send the new delegate instance as a parameter to the `Thread` constructor. In F#, if a function expects a delegate as its parameter, a lambda expression or a function can be sent and the compiler will behave as if you sent the delegate. So, in the above code, the function `myConMethod` is sent as the parameter to the `Thread` constructor, but what is actually sent is a new instance of `ThreadStart` (to which was sent `myConMethod`).

The `Thread` class defines the `Sleep` method, which puts the thread from which it is called to sleep for the number of milliseconds that is sent to it as a parameter.

Shared immutable data does not require synchronization among the threads that access it. However, if the shared data is mutable, which is possible in F#, locking will be required to prevent corruption of the shared data by multiple threads attempting to change it. A mutable variable can be locked while a function operates on it to provide synchronized access to the object with the `lock` function. This function takes two parameters, the first of which is the variable to be changed. The second parameter is a lambda expression that changes the variable.

A mutable heap-allocated variable is of type `ref`. For example, the following declaration creates such a variable named `sum` with the initial value of 0:

```
let sum = ref 0
```

A `ref` type variable can be changed in a lambda expression that uses the ALGOL/Pascal/Ada assignment operator, `:=`. The `ref` variable must be prefixed with an exclamation point (!) to get its value. In the following, the mutable variable `sum` is locked while the lambda expression adds the value of `x` to it:

```
lock(sum) (fun () -> sum := !sum + x)
```

Threads can be called asynchronously, just as with C#, using the same subprograms, `BeginInvoke` and `EndInvoke`, as well as the `IAsyncResult` interface to facilitate the determination of the completion of the execution of the asynchronously called thread.

As stated previously, F# has the concurrent generic collections of .NET available to its programs. This can save a great deal of programming effort when building multithreaded programs that need a shared data structure in the form of a queue, stack, or bag.

13.10 Statement-Level Concurrency

In this section, we take a brief look at language design for statement-level concurrency. From the language design point of view, the objective of such designs is to provide a mechanism that the programmer can use to inform the compiler of ways it can map the program onto a multiprocessor architecture.⁹

In this section, only one collection of linguistic constructs from one language for statement-level concurrency is discussed: High-Performance Fortran.

13.10.1 High-Performance Fortran

High-Performance Fortran (HPF; ACM, 1993b) is a collection of extensions to Fortran 90 that are meant to allow programmers to specify information to the compiler to help it optimize the execution of programs on multiprocessor computers. HPF includes both new specification statements and intrinsic, or built-in, subprograms. This section discusses only some of the HPF statements.

The primary specification statements of HPF are for specifying the number of processors, the distribution of data over the memories of those processors, and the alignment of data with other data in terms of memory placement. The HPF specification statements appear as special comments in a Fortran program. Each of them is introduced by the prefix `!HPF$`, where `!` is the character used to begin lines of comments in Fortran 90. This prefix makes them invisible to Fortran 90 compilers but easy for HPF compilers to recognize.

The `PROCESSORS` specification has the following form:

```
!HPF$ PROCESSORS procs (n)
```

This statement is used to specify to the compiler the number of processors that can be used by the code generated for this program. This information is used in conjunction with other specifications to tell the compiler how data are to be distributed to the memories associated with the processors.

The `DISTRIBUTE` and `ALIGN` specifications are used to provide information to the compiler on machines that do not share memory—that is, each processor has its own memory. The assumption is that an access by a processor to its own memory is faster than an access to the memory of another processor.

9. Although ALGOL 68 included a semaphore type that was meant to deal with statement-level concurrency, we do not discuss that application of semaphores here.

The `DISTRIBUTE` statement specifies what data are to be distributed and the kind of distribution that is to be used. Its form is as follows:

```
!HPF$ DISTRIBUTE (kind) ONTO procs :: identifier_list
```

In this statement, `kind` can be either `BLOCK` or `CYCLIC`. The identifier list is the names of the array variables that are to be distributed. A variable that is specified to be `BLOCK` distributed is divided into n equal groups, where each group consists of contiguous collections of array elements evenly distributed over the memories of all the processors. For example, if an array with 500 elements named `LIST` is `BLOCK` distributed over 5 processors, the first 100 elements of `LIST` will be stored in the memory of the first processor, the second 100 in the memory of the second processor, and so forth. A `CYCLIC` distribution specifies that individual elements of the array are cyclically stored in the memories of the processors. For example, if `LIST` is `CYCLIC` distributed, again over five processors, the first element of `LIST` will be stored in the memory of the first processor, the second element in the memory of the second processor, and so forth.

The form of the `ALIGN` statement is

```
ALIGN array1_element WITH array2_element
```

`ALIGN` is used to relate the distribution of one array with that of another. For example,

```
ALIGN list1(index) WITH list2(index+1)
```

specifies that the `index` element of `list1` is to be stored in the memory of the same processor as the `index+1` element of `list2`, for all values of `index`. The two array references in an `ALIGN` appear together in some statement of the program. Putting them in the same memory (which means the same processor) ensures that the references to them will be as close as possible.

Consider the following example code segment:

```
REAL list_1 (1000), list_2 (1000)
INTEGER list_3 (500), list_4 (501)
!HPF$ PROCESSORS proc (10)
!HPF$ DISTRIBUTE (BLOCK) ONTO procs :: list_1, list_2
!HPF$ ALIGN list_3 (index) WITH list_4 (index+1)
. . .
list_1 (index) = list_2 (index)
list_3 (index) = list_4 (index+1)
```

In each execution of these assignment statements, the two referenced array elements will be stored in the memory of the same processor.

The HPF specification statements provide information for the compiler that it may or may not use to optimize the code it produces. What the compiler actually does depends on its level of sophistication and the particular architecture of the target machine.

The `FORALL` statement specifies a sequence of assignment statements that may be executed concurrently. For example,

```
FORALL (index = 1:1000)
  list_1(index) = list_2(index)
END FORALL
```

specifies the assignment of the elements of `list_2` to the corresponding elements of `list_1`. However, the assignments are restricted to the following order: the right side of all 1,000 assignments must be evaluated first, before any assignments take place. This permits concurrent execution of all of the assignment statements. In addition to assignment statements, `FORALL` statements can appear in the body of a `FORALL` construct. The `FORALL` statement is a good match with vector machines, in which the same instruction is applied to many data values, usually in one or more arrays. The HPF `FORALL` statement is included in Fortran 95 and subsequent versions of Fortran.

We have briefly discussed only a small part of the capabilities of HPF. However, it should be enough to provide the reader with an idea of the kinds of language extensions that are useful for programming computers with possibly large numbers of processors.

C# 4.0 (and the other .NET languages) include two methods that behave somewhat like `FORALL`. They are loop control statements in which the iterations can be unrolled and the bodies executed concurrently. These are `Parallel.For` and `Parallel.ForEach`.

S U M M A R Y

Concurrent execution can be at the instruction, statement, or subprogram level. We use the phrase *physical concurrency* when multiple processors are actually used to execute concurrent units. If concurrent units are executed on a single processor, we use the term *logical concurrency*. The underlying conceptual model of all concurrency can be referred to as *logical concurrency*.

Most multiprocessor computers fall into one of two broad categories—SIMD or MIMD. MIMD computers can be distributed.

Languages that support subprogram-level concurrency must provide two fundamental capabilities: mutually exclusive access to shared data structures (competition synchronization) and cooperation among tasks (cooperation synchronization).

Tasks can be in any one of five different states: new, ready, running, blocked, or dead.

Rather than designing language constructs for supporting concurrency, sometimes libraries, such as OpenMP, are used.

The design issues for language support for concurrency are how competition and cooperation synchronization are provided, how an application can

influence task scheduling, how and when tasks start and end their executions, and how and when they are created.

A semaphore is a data structure consisting of an integer and a task description queue. Semaphores can be used to provide both competition and cooperation synchronization among concurrent tasks. It is easy to use semaphores incorrectly, resulting in errors that cannot be detected by the compiler, linker, or run-time system.

Monitors are data abstractions that provide a natural way of providing mutually exclusive access to data shared among tasks. They are supported by several programming languages, among them Ada, Java, and C#. Cooperation synchronization in languages with monitors must be provided with some form of semaphores.

The underlying concept of the message-passing model of concurrency is that tasks send each other messages to synchronize their execution.

Ada provides complex but effective constructs, based on the message-passing model, for concurrency. Ada's tasks are heavyweight tasks. Tasks communicate with each other through the rendezvous mechanism, which is synchronous message passing. A rendezvous is the action of a task accepting a message sent by another task. Ada includes both simple and complicated methods of controlling the occurrences of rendezvous among tasks.

Ada 95+ includes additional capabilities for the support of concurrency, primarily protected objects. Ada 95+ supports monitors in two ways, with tasks and with protected objects.

Java supports lightweight concurrent units in a relatively simple but effective way. Any class that either inherits from `Thread` or implements `Runnable` can override a method named `run` and have that method's code executed concurrently with other such methods and with the main program. Competition synchronization is specified by defining methods that access shared data to be implicitly synchronized. Small sections of code can also be implicitly synchronized. A class whose methods are all synchronized is a monitor. Cooperation synchronization is implemented with the methods `wait`, `notify`, and `notifyAll`. The `Thread` class also provides the `sleep`, `yield`, `join`, and `interrupt` methods.

Java has direct support for counting semaphores through its `Semaphore` class and its `acquire` and `release` methods. It also had some classes for providing nonblocking atomic operations, such as addition, increment, and decrement operations for integers. Java also provides explicit locks with the `Lock` interface and `ReentrantLock` class and its `lock` and `unlock` methods. In addition to implicit synchronization using `synchronized`, Java provides implicit nonblocking synchronization of `int`, `long`, and `boolean` type variables, as well as references and arrays. In these cases, atomic getters, setters, add, increment, and decrement operations are provided.

C#'s support for concurrency is based on that of Java but is slightly more sophisticated. Any method can be run in a thread. Both actor and server threads are supported. All threads are controlled through associated delegates. Server threads can be synchronously called with `Invoke` or asynchronously called

with `BeginInvoke`. A callback method address can be sent to the called thread. Three kinds of thread synchronization are supported with the `Interlocked` class, which provides atomic increment and decrement operations, the `Monitor` class, and the `lock` statement.

All .NET languages have the use of the generic concurrent data structures for stacks, queues, and bags, for which competition synchronization is implicit.

Multi-LISP extends Scheme slightly to allow the programmer to inform the implementation about program parts that can be executed concurrently. Concurrent ML extends ML to support a form of threads and a form of synchronous message passing among those threads. This message passing is designed with channels. F# programs have access to all of the .NET support classes for concurrency. Data shared among threads that is mutable can have access synchronized.

High-Performance Fortran includes statements for specifying how data is to be distributed over the memory units connected to multiple processors. Also included are statements for specifying collections of statements that can be executed concurrently.

BIBLIOGRAPHIC NOTES

The general subject of concurrency is discussed at great length in Andrews and Schneider (1983), Holt et al. (1978), and Ben-Ari (1982).

The monitor concept is developed and its implementation in Concurrent Pascal is described by Brinch Hansen (1977).

The early development of the message-passing model of concurrent unit control is discussed by Hoare (1978) and Brinch Hansen (1978). An in-depth discussion of the development of the Ada tasking model can be found in Ichbiah et al. (1979). Ada 95 is described in detail in ARM (1995). High-Performance Fortran is described in ACM (1993b).

REVIEW QUESTIONS

1. What are the three possible levels of concurrency in programs?
2. Describe the logical architecture of an SIMD computer.
3. Describe the logical architecture of an MIMD computer.
4. What level of program concurrency is best supported by SIMD computers?
5. What level of program concurrency is best supported by MIMD computers?
6. Describe the logical architecture of a vector processor.
7. What is the difference between physical and logical concurrency?

8. What is a thread of control in a program?
9. Why are coroutines called quasi-concurrent?
10. What is a multithreaded program?
11. What are four reasons for studying language support for concurrency?
12. What is a heavyweight task? What is a lightweight task?
13. Define *task*, *synchronization*, *competition* and *cooperation synchronization*, *liveness*, *race condition*, and *deadlock*.
14. What kind of tasks do not require any kind of synchronization?
15. Describe the five different states in which a task can be.
16. What is a task descriptor?
17. In the context of language support for concurrency, what is a guard?
18. What is the purpose of a task-ready queue?
19. What are the two primary design issues for language support for concurrency?
20. Describe the actions of the wait and release operations for semaphores.
21. What is a binary semaphore? What is a counting semaphore?
22. What are the primary problems with using semaphores to provide synchronization?
23. What advantage do monitors have over semaphores?
24. In what three common languages can monitors be implemented?
25. Define *rendezvous*, **accept clause**, **entry clause**, *actor task*, *server task*, *extended accept clause*, *open accept clause*, *closed accept clause*, and *completed task*.
26. Which is more general, concurrency through monitors or concurrency through message passing?
27. Are Ada tasks created statically or dynamically?
28. What purpose does an extended **accept** clause serve?
29. How is cooperation synchronization provided for Ada tasks?
30. What is the advantage of protected objects in Ada 95 over tasks for providing access to shared data objects?
31. Specifically, what Java program unit can run concurrently with the main method in an application program?
32. Are Java threads lightweight or heavyweight tasks?
33. What does the Java `sleep` method do?
34. What does the Java `yield` method do?
35. What does the Java `join` method do?
36. What does the Java `interrupt` method do?
37. What are the two Java constructs that can be declared to be synchronized?

38. How can the priority of a thread be set in Java?
39. Can Java threads be actor threads, server threads, or either?
40. Describe the actions of the three Java methods that are used to support cooperation synchronization.
41. What kind of Java object is a monitor?
42. Explain why Java includes the `Runnable` interface.
43. What are the two methods used with Java `Semaphore` objects?
44. What is the advantage of the nonblocking synchronization in Java?
45. What are the methods of the Java `AtomicInteger` class and what is the purpose of this class?
46. How are explicit locks supported in Java?
47. What kinds of methods can be run in a C# thread?
48. Can C# threads be actor threads, server threads, or either?
49. What are the two ways a C# thread can be called synchronously?
50. How can a C# thread be called asynchronously?
51. How is the returned value from an asynchronously called thread retrieved in C#?
52. What is different about C#'s `Sleep` method, relative to Java's `sleep`?
53. What exactly does C#'s `Abort` method do?
54. What is the purpose of C#'s `Interlocked` class?
55. What does the C# `lock` statement do?
56. On what language is Multi-LISP based?
57. What is the semantics of Multi-LISP's `pcall` construct?
58. How is a thread created in CML?
59. What is the type of an F# heap-allocated mutable variable?
60. Why don't F# immutable variables require synchronized access in a multithreaded program?
61. What is the objective of the specification statements of HPF?
62. What is the purpose of the `FORALL` statement of HPF and Fortran?

PROBLEM SET

1. Explain clearly why competition synchronization is not a problem in a programming environment that supports coroutines but not concurrency.
2. What is the best action a system can take when deadlock is detected?
3. Busy waiting is a method whereby a task waits for a given event by continuously checking for that event to occur. What is the main problem with this approach?

4. In the producer-consumer example of Section 13.3, suppose that we incorrectly replaced the `release(access)` in the consumer process with `wait(access)`. What would be the result of this error on execution of the system?
5. From a book on assembly language programming for a computer that uses an Intel Pentium processor, determine what instructions are provided to support the construction of semaphores.
6. Suppose two tasks, A and B, must use the shared variable `Buf_Size`. Task A adds 2 to `Buf_Size`, and task B subtracts 1 from it. Assume that such arithmetic operations are done by the three-step process of fetching the current value, performing the arithmetic, and putting the new value back. In the absence of competition synchronization, what sequences of events are possible and what values result from these operations? Assume that the initial value of `Buf_Size` is 6.
7. Compare the Java competition synchronization mechanism with that of Ada.
8. Compare the Java cooperation synchronization mechanism with that of Ada.
9. What happens if a monitor procedure calls another procedure in the same monitor?
10. Explain the relative safety of cooperation synchronization using semaphores and using Ada's **when** clauses in tasks.

PROGRAMMING EXERCISES

1. Write an Ada task to implement general semaphores.
2. Write an Ada task to manage a shared buffer such as the one in our example, but use the semaphore task from Programming Exercise 1.
3. Define semaphores in Ada and use them to provide both cooperation and competition synchronization in the shared-buffer example.
4. Write Programming Exercise 3 using Java.
5. Write the shared-buffer example of the chapter in C#.
6. The reader-writer problem can be stated as follows: A shared memory location can be concurrently read by any number of tasks, but when a task must write to the shared memory location, it must have exclusive access. Write a Java program for the reader-writer problem.
7. Write Programming Exercise 6 using Ada.
8. Write Programming Exercise 6 using C#.

